College0

Phase II: Design Report

Part 1: Introduction — Collaboration Class Diagram

The collaboration class diagram below provides an overall picture of the College0 system. It shows all major domain classes, their key attributes and methods, and the relationships between them. This diagram serves as the structural foundation from which all use cases and design decisions in Part 2 are derived.

1.1 System Purpose

College0 is a local GUI application that manages the full academic lifecycle of a small college program: admissions, course registration, semester management, grading, disciplinary actions, and an AI-powered Q&A assistant. The system supports four roles (Visitor, Student, Instructor, Registrar) and integrates a local vector database with an Ollama-hosted LLM for the AI feature.

1.2 Class Inventory

The system is built from ten primary classes organized into three tiers:

- User Tier (abstract + subclasses): User, Student, Instructor, Registrar
- Domain Tier (core business objects): Course, Enrollment, Review, Complaint
- Infrastructure Tier (services + state): Semester, AIAssistant

1.3 Relationship Summary

From Class	To Class	Type	Meaning
User	Student	Inheritance	Student is a specialised User with academic attributes
User	Instructor	Inheritance	Instructor is a specialised User who teaches courses
User	Registrar	Inheritance	Registrar is a super-user with full system authority
Student	Enrollment	Association	A student can have many enrollment records (1..*)
Course	Enrollment	Association	A course holds many enrollment records (1..*)
Student	Review	Association	A student writes reviews for enrolled courses
Review	Course	Association	Each review rates exactly one course
Instructor	Course	Association	An instructor teaches one or more courses per semester

From Class	To Class	Type	Meaning
Course	Semester	Association	Every course belongs to exactly one semester
Registrar	Semester	Association	Registrar controls semester period transitions
Student	Complaint	Association	Students and instructors file complaints
Registrar	Complaint	Association	Registrar arbitrates and resolves all complaints
All Users	AIAssistant	Uses	All authenticated roles can query the AI assistant

1.4 Class Descriptions

User (Abstract Base)

The root of the user hierarchy. Holds shared identity and security attributes shared by all three user roles. Never instantiated directly.

Attribute / Method	Description
userId : String	Unique system-generated identifier
role : Role	Enum: VISITOR | STUDENT | INSTRUCTOR | REGISTRAR
passwordHash : String	Bcrypt hash; plaintext never stored
warnings : int	Count of active warnings; 3 triggers suspension
suspended : bool	True if the user is currently suspended from the system
login()	Validates credentials; enforces first-login password change
changePassword()	Hashes and persists new password; clears first-login flag
receiveWarning(reason)	Increments warnings, persists reason, checks suspension threshold

Student

Extends User. Manages the full academic record of a matriculated student including GPA, registrations, disciplinary status, and graduation eligibility.

Attribute / Method	Description
gpa : float	Cumulative GPA on 4.0 scale, rounded to 2 decimal places
semGPA : float	Current semester GPA; recalculated at close of grading period
honorsCount : int	Number of Honor Roll distinctions earned; each can clear one warning
completedCourses : List	IDs of all courses passed (grade D or above); used for graduation check

Attribute / Method	Description
status : Status	Enum: ACTIVE | SUSPENDED | TERMINATED | GRADUATED
register(courseId)	Attempts enrolment; checks period, conflict, capacity, pass history
submitReview(courseId, text, stars)	Submits review; applies taboo filter, publishes or suppresses
applyGraduation()	Checks 8 completed courses; submits graduation request to Registrar

Instructor

Extends User. Manages teaching assignments, student grading, and waitlist decisions.

Attribute / Method	Description
instructorId	Unique instructor identifier, separate from userId
qualifications : String	Free-text qualifications submitted during application
suspendedNextSem : bool	Set true if all courses were cancelled; bars next-semester assignment
gradeStudent(studentId, courseId, grade)	Assigns letter grade; validates student is enrolled in the course
manageWaitlist(courseId)	Promotes or rejects waitlisted students; increments enrolled count
fileComplaint(subjectId, desc)	Files complaint requesting warning or de-registration against a student

Registrar

Extends User with adminLevel = SUPER. Has unrestricted access to all system data and operations.

Attribute / Method	Description
adminLevel = SUPER	Distinguishes Registrar from other User subclasses in RBAC checks
managePeriod()	Advances semester state: SETUP→REGISTRATION→RUNNING→GRADING→CLOSED
resolveComplaint(id, action)	Issues warning, enforces de-reg, or dismisses; action is mandatory for instructor complaints
manageTaboo()	Adds/removes words from the taboo list used by the review filter

Course

Represents a single class offering within a semester. Tracks enrolment, ratings, and cancellation state.

Attribute / Method	Description
courseId	Unique course identifier

Attribute / Method	Description
name, description	Human-readable course name and full description
timeSlot : String	Registrar-assigned slot; used for conflict detection
maxSize : int	Maximum enrolled students; excess students go to waitlist
enrolledCount : int	Current number of fully enrolled (non-waitlisted) students
avgRating : float	Live average of all published star ratings (1.0–5.0)
cancelled : bool	True if the course was cancelled due to under-enrolment (<3 students)
addStudent(studentId)	Enrols student or adds to waitlist; updates enrolledCount
cancelIfUnderEnrolled()	Cancels course if enrolledCount < 3 at start of RUNNING period
getAvgRating()	Recomputes average from all published Review records

Enrollment (Association Class)

Junction class between Student and Course. Holds the grade and enrolment status for a single student-course pairing.

Attribute / Method	Description
enrollmentId	Primary key
grade : Grade	Enum: A | B | C | D | F | NULL (null until graded)
status : EnrollStatus	Enum: ENROLLED | WAITLISTED | DEREGISTERED | COMPLETED
promote()	Moves student from WAITLISTED to ENROLLED; increments enrolledCount
deregister()	Sets status to DEREGISTERED; decrements enrolledCount

Review

A student-written rating and text review for a course. Subject to taboo-word filtering before publication.

Attribute / Method	Description
reviewId	Unique identifier
text : String	Full review body; may be redacted before publication
starRating : int	Integer 1–5; contributes to course avgRating
redacted : bool	True if taboo words were replaced with *
published : bool	False if review was rejected (3+ taboo hits)
authorId : String	Visible only to Registrar for identity attribution
checkTaboo(tabooList)	Case-insensitive partial match; returns violation count
publish()	Marks published=true; triggers avgRating recalculation on Course

Attribute / Method	Description
redact()	Replaces all taboo matches with *; sets redacted=true

Semester

Tracks the active administrative period and controls what operations are permitted across the whole system. Acts as a global state machine.

Attribute / Method	Description
semesterId	Unique identifier for the semester instance
period : Period	Enum: SETUP | REGISTRATION | RUNNING | GRADING | CLOSED
year : int	Academic year of this semester
quota : int	Maximum student enrolment capacity for the entire program
advancePeriod()	Moves to next state in sequence; throws if transition is invalid
closeSemester()	Finalises academic records; triggers GPA recalc and standing checks

Complaint

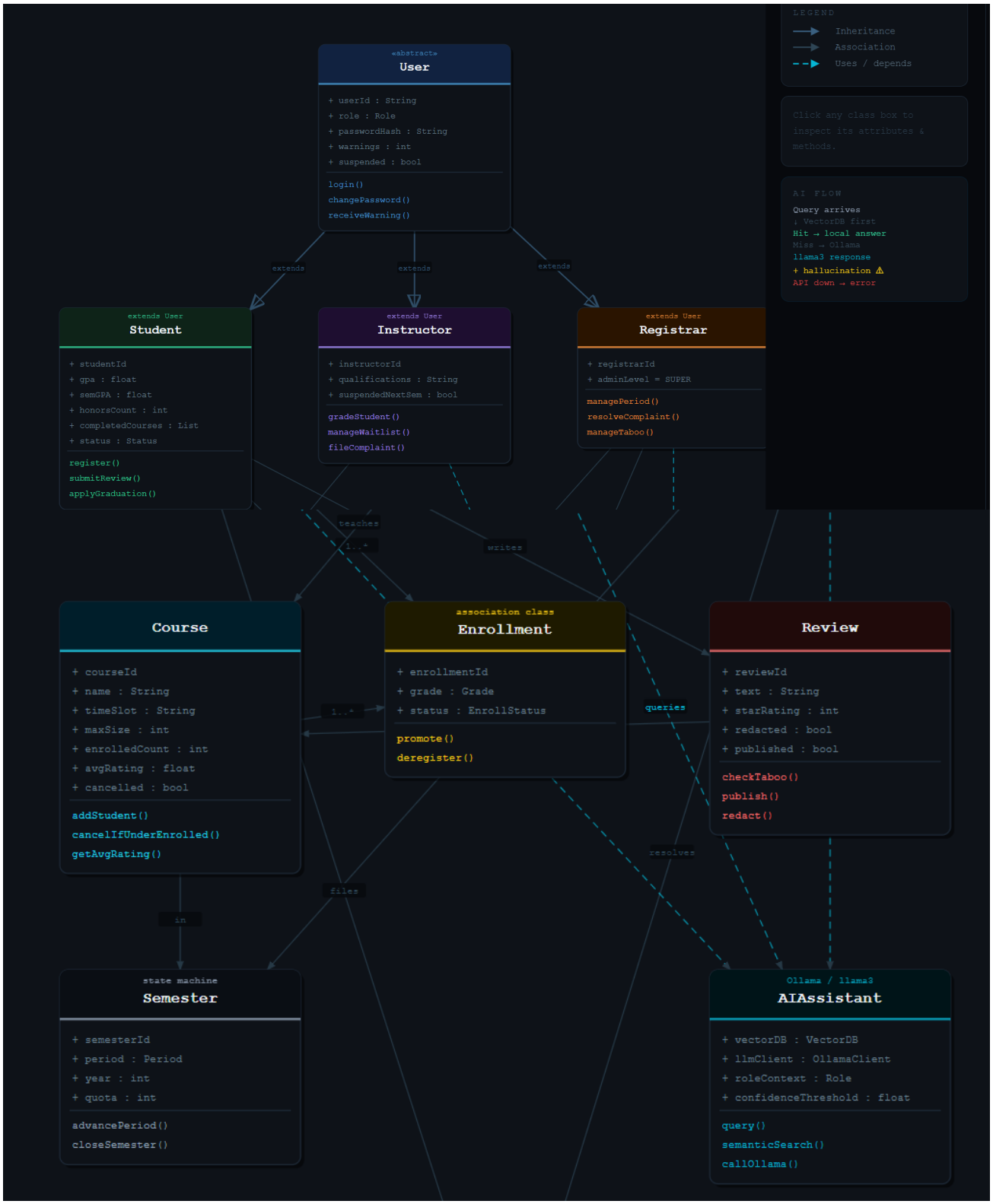
Records disciplinary complaints filed by students or instructors and tracks Registrar resolution.

Attribute / Method	Description
complaintId	Unique identifier
filerId : String	userId of the person filing the complaint
subjectId : String	userId of the person being complained about
type : ComplaintType	Enum: STUDENT_VS_STUDENT | STUDENT_VS_INSTRUCTOR | INSTRUCTOR_VS_STUDENT
status : Status	Enum: OPEN | UNDER_REVIEW | RESOLVED
resolution : String	Registrar notes; mandatory for instructor-initiated complaints
file()	Persists complaint with status=OPEN; adds to Registrar queue
resolve(action)	Applies action (warn/deregister/dismiss); closes complaint
escalate()	Flags complaint for priority review if no action within 48 h

AIAssistant

Singleton service accessible from every authenticated user's personal page. Implements a two-stage retrieval pipeline: local vector database first, Ollama LLM on cache miss.

Attribute / Method	Description
vectorDB : VectorDB	Local embedded vector store pre-populated with college policies
llmClient : OllamaClient	HTTP client targeting Ollama at localhost:11434
model : String	Default 'llama3'; fallback 'phi3' for low-memory environments
confidenceThreshold	Minimum cosine similarity (0.75) to accept a local VectorDB result
query(text, role, userId)	Entry point: builds role context, calls semanticSearch, routes result
semanticSearch(text)	Embeds query, searches VectorDB; returns result + confidence score
callOllama(prompt)	POSTs to /api/chat; attaches hallucination warning to response


«abstract»
User
+ userId : String
+ role : Role
+ passwordHash : String
+ warnings : int
+ suspended : bool
login()
changePassword()
receiveWarning()
extends
extends
extends
extends User
Student
+ studentId
+ gpa : float
+ semGPA : float
+ honorsCount : int
+ completedCourses : List
+ status : Status
register()
submitReview()
applyGraduation()
extends User
Instructor
+ instructorId
+ qualifications : String
+ suspendedNextSem : bool
gradeStudent()
manageWaitlist()
fileComplaint()
extends User
Registrar
+ registrarId
+ adminLevel = SUPER
managePeriod()
resolveComplaint()
manageTaboo()
teaches
1..*
writes
Course
+ courseId
+ name : String
+ timeSlot : String
+ maxSize : int
+ enrolledCount : int
+ avgRating : float
+ cancelled : bool
addStudent()
cancelIfUnderEnrolled()
getAvgRating()
1..*
association class
Enrollment
+ enrollmentId
+ grade : Grade
+ status : EnrollStatus
promote()
deregister()
queries
Review
+ reviewId
+ text : String
+ starRating : int
+ redacted : bool
+ published : bool
checkTaboo()
publish()
redact()
resolves
files
in
state machine
Semester
+ semesterId
+ period : Period
+ year : int
+ quota : int
advancePeriod()
closeSemester()
Ollama / llama3
AIAssistant
+ vectorDB : VectorDB
+ llmClient : OllamaClient
+ roleContext : Role
+ confidenceThreshold : float
query()
semanticSearch()
callOllama()
LEGEND
Inheritance
Association
Uses / depends
Click any class box to inspect its attributes & methods.
AI FLOW
Query arrives
↓ VectorDB first
Hit → local answer
Miss → Ollama
llama3 response
+ hallucination ⚠
API down → error

Part 2: All Use Cases — Scenarios & Petri Nets

This section documents all ten use cases of College0. For each use case the following are provided:

- Actor(s) and preconditions
- Normal scenario (step-by-step)
- All exceptional / alternative scenarios
- Postconditions
- Petri net description: places, transitions, arcs, and initial marking

> *Petri Net Notation: Circles = Places (states/conditions). Rectangles = Transitions (events/actions). Arrows = Arcs (flow of tokens). A filled circle inside a place = token (active state). A transition fires when all its input places hold a token.*

UC-01: View Public Dashboard

Actors: All users (Visitor, Student, Instructor, Registrar)

Preconditions: Application has been launched. No authentication required.

Normal Scenario

- System launches and renders the main window without requiring login.
- DashboardController.load() fires three parallel data queries:
  - CourseDAO.getTopRated(3) — returns courses sorted descending by avgRating.
  - CourseDAO.getBottomRated(3) — returns courses sorted ascending by avgRating.

    - StudentDAO.getTopGPA(3) — returns students sorted descending by cumulative GPA.
- Results are rendered in three card panels on the dashboard.
- A login form is presented in the top navigation bar for returning users.
- Authenticated users who log in see additional role-appropriate navigation links without the page reloading.

Exceptional Scenarios

- E1 — No courses rated yet: rating panels display 'No ratings available yet.'
- E2 — No students enrolled: GPA panel displays 'No student records found.'
- E3 — Database unavailable on launch: all panels show an error banner with a Retry button; the application does not crash.

Postconditions

- The user sees publicly available program information.
- The page reflects live data — any subsequent change to ratings or GPA is reflected on next load.

Petri Net — UC-01: Public Dashboard Load

> *This net models the parallel data-fetch and render flow triggered on application launch.*

Element	Description
PLACES	P0: App launched | P1: DB available | P2: DB unavailable | P3: Top-rated fetched | P4: Bottom-rated fetched | P5: Top-GPA fetched | P6: Dashboard rendered | P7: Error shown
TRANSITIONS	T0: Check DB → P1 or P2 | T1: Fetch top-rated | T2: Fetch bottom-rated | T3: Fetch top-GPA | T4: Merge results + render | T5: Show error + retry
INITIAL MARKING	Token in P0 (app launched). T0 fires immediately; on success: token moves to P1 and T1/T2/T3 fire in parallel. On failure: token in P2, T5 fires.
KEY LOGIC	T4 (Merge + Render) requires tokens in P3, P4, and P5 simultaneously — this is the synchronisation/join transition. All three fetches must complete before the dashboard is drawn.

```
  P0(App Launched) ——T0——> [DB Check]
                                |————————————> P1(DB OK) ——> T1 ——> P3(Top-rated) ——\
                                |                        ——> T2 ——> P4(Bot-rated) ——+——> T4
——> P6(Dashboard Rendered)
                                |                        ——> T3 ——> P5(Top-GPA)   ——/
                                |————————————> P2(DB Error) ——> T5 ——> P7(Error Banner)
```

UC-02: Visitor Application (Student or Instructor)

Actors: Visitor (applicant), Registrar (decision-maker)

Preconditions: User is unauthenticated. Registration period has not been finalised.

Normal Scenario

- Visitor selects 'Apply as Student' or 'Apply as Instructor' from the public dashboard.
- Visitor fills in the application form. Student applications require academic GPA; instructor applications require qualifications text.
- ApplicationService.submit() validates required fields and persists an Application record with status = PENDING.
- For student applications: system auto-evaluates eligibility. If GPA > 3.0 AND enrolledCount < quota → flag RECOMMENDED_ACCEPT; otherwise RECOMMENDED_REJECT.
- Registrar reviews the pending queue and makes a decision.
- If the Registrar's decision contradicts the system recommendation for a student: a justification message field becomes mandatory.
- On approval as student: CredentialService generates a unique studentId and a one-time password; credentials are sent to the applicant.
- On approval as instructor: Registrar assigns instructor to course(s) during Class Setup Period.
- On rejection: applicant is notified. Student rejections that override the recommendation include the Registrar's justification.

Exceptional Scenarios

- E1 — Quota full: any student application is auto-flagged QUOTA_EXCEEDED regardless of GPA.
- E2 — Incomplete form: system prevents submission; highlights missing required fields.
- E3 — Registrar overrides recommendation without justification: form submission is blocked until justification is written.
- E4 — Instructor rejection: no GPA check or justification required.

Postconditions

- Applicant is either admitted (credentials issued) or rejected (notified with optional justification).
- Application queue is updated. Audit log records the decision and any override justification.

Petri Net — UC-02: Visitor Application

> *This net models the dual-path (student vs instructor) application and decision flow.*

Element	Description
PLACES	P0: Visitor idle | P1: Form filled (student) | P2: Form filled (instructor) | P3: PENDING (student) | P4: PENDING (instructor) | P5: Auto-eval done | P6: Quota exceeded | P7:

Element	Description
	Registrar reviewing | P8: Approved (student) | P9: Approved (instructor) | P10: Rejected | P11: Override requires justification
TRANSITIONS	T0: Submit student app | T1: Submit instructor app | T2: Auto-evaluate GPA+quota | T3: Quota check fails | T4: Registrar approves (agrees rec) | T5: Registrar approves (overrides) | T6: Registrar rejects | T7: Enter justification | T8: Issue credentials
INITIAL MARKING	Token in P0. After form submission, one token enters either P3 or P4 depending on application type. Student path additionally fires T2 to produce a recommendation token in P5 or P6.
KEY LOGIC	T5 (Registrar approves against recommendation) requires token in P5 AND produces token in P11 (justification required). T4 can only fire if P5 has recommendation=ACCEPT. This models the mandatory-justification constraint.

```
  P0(Visitor) ──T0──> P3(Pending-Student) ──T2──> P5(Eval OK) ──T4(agree)──>
P7(Reviewing) ──T8──> P8(Approved-Student)
                                          └──T3──> P6(Quota Full) ──────────> P7
──T6──> P10(Rejected)
             ──T1──> P4(Pending-Instructor) ──────────────────────────────> P7
──T6──> P10
  T5(override) requires P5 token AND mandatory P11(Justification) before P7
```

UC-03: Semester Period Management

Actors: Registrar (period controller). All roles are affected.

Preconditions: Registrar is authenticated with admin access.

Normal Scenario

- Registrar initiates the SETUP period: creates courses with name, description, time slot, instructor assignment, and enrolment cap.
- Registrar advances to REGISTRATION: students may now register for courses.
- Registrar advances to RUNNING: general registration locks. RunningPeriodJob executes automatically (see UC-05).
- Registrar advances to GRADING: instructors submit grades for all enrolled students.
- Registrar closes the semester (CLOSED): GPA recalculated, Honor Roll awarded, warnings issued, terminations processed.

Exceptional Scenarios

- E1 — Attempt to skip a period: SemesterService.advancePeriod() throws IllegalTransitionException; UI shows an error.
- E2 — Attempt to reverse a period: same exception; periods cannot go backwards.
- E3 — Registrar tries to close semester with ungraded students: system warns but allows closure; ungraded instructors receive warnings.

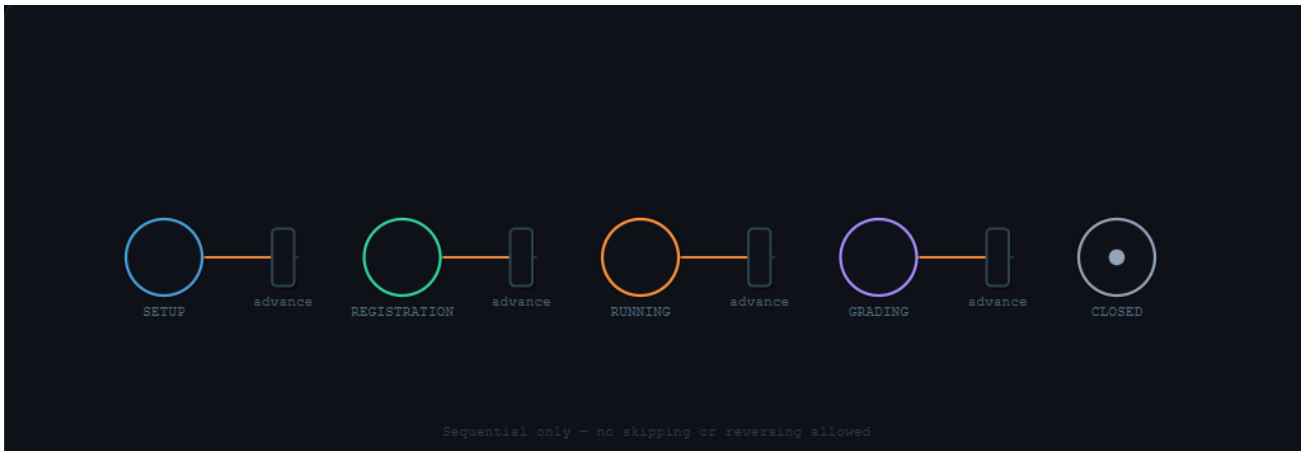
Postconditions

- The system is in the next period. Role-based permissions are updated immediately across all active sessions.

Petri Net — UC-03: Semester Period State Machine

> *This net is a strict sequential state machine. The inhibitor arc from each period to its predecessor enforces the no-reversal rule.*

Element	Description
PLACES	P0: SETUP | P1: REGISTRATION | P2: RUNNING | P3: GRADING | P4: CLOSED | Perr: Invalid transition
TRANSITIONS	T0: Advance SETUP→REGISTRATION | T1: Advance REGISTRATION→RUNNING (+ fires RunningPeriodJob) | T2: Advance RUNNING→GRADING | T3: Advance GRADING→CLOSED (+ fires ClosingSemesterJob) | Terr: Illegal transition attempted
ARCS	Each Ti takes token from Pi and deposits into Pi+1. Inhibitor arcs from each Pi prevent any Ti-1 from firing (no reverse). Terr fires only when no valid transition is enabled.
INITIAL MARKING	One token in P0 (SETUP). Exactly one token exists in the net at all times — this enforces mutual exclusion: exactly one period is active.
KEY LOGIC	T1 has a side-effect arc to a RunningPeriodJob place (triggers under-enrolment checks and course cancellation). T3 has a side-effect arc to ClosingSemesterJob (triggers GPA recalc, standing checks).



UC-04: Course Registration

Actors: Student (registrant), Instructor (waitlist manager), Registrar (administrator)

Preconditions: System is in REGISTRATION period. Student is authenticated and matriculated.

Normal Scenario

- Student views the list of available courses for the current semester.
- Student selects courses (minimum 2, maximum 4 per semester).
- RegistrationService.register(studentId, courseId) is called for each selection.
- System checks: (a) period == REGISTRATION, (b) enrolment count 2–4, (c) no time-slot conflict, (d) course not already passed.
- If course not full (enrolledCount < maxSize): EnrollmentRecord created with status = ENROLLED; enrolledCount incremented.
- If course full: WaitlistRecord created with status = WAITLISTED.
- Instructor reviews waitlist and may manually promote students using manageWaitlist().
- Students may adjust course selections any number of times during REGISTRATION, subject to all constraints.

Exceptional Scenarios

- E1 — Time conflict: RegistrationService throws TimeConflictException; existing schedule is shown to the student.
- E2 — Max courses (4) reached: system blocks additional registration with a quota message.
- E3 — Course already passed (D or above): system displays a warning dialog; Registrar can override to allow re-registration.
- E4 — Course previously failed (F): re-registration is permitted without any warning.
- E5 — Student attempts to drop below 2 courses after the minimum is met: system warns but permits (< 2 at start of RUNNING triggers a warning per UC-05).

Postconditions

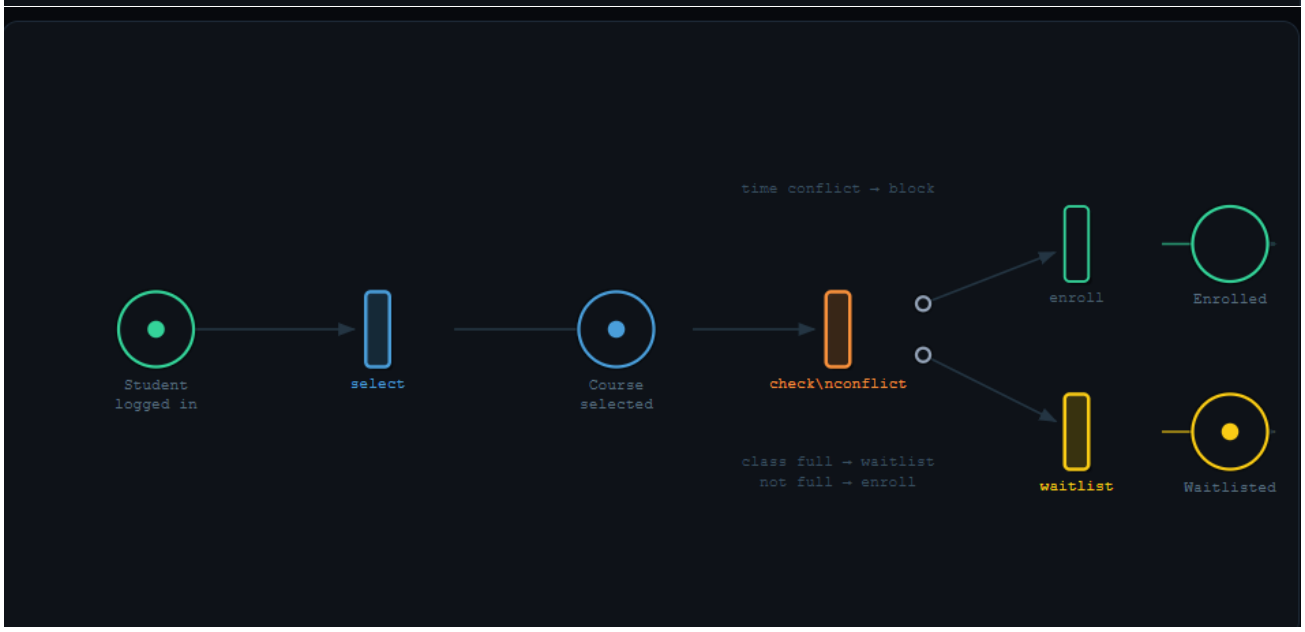
- Student is enrolled in selected courses or placed on applicable waitlists. Enrollment records are persisted.

Petri Net — UC-04: Course Registration

> *This net models the full registration decision tree including conflict detection, capacity check, and waitlist branching.*

Element	Description
PLACES	P0: Student logged in | P1: Course selected | P2: Period check passed | P3: Period check failed | P4: Conflict check passed | P5: Conflict detected | P6: Capacity check passed | P7: Class full | P8: Enrolled | P9: Waitlisted | P10: Waitlist promoted | P11: Max courses reached
TRANSITIONS	T0: Select course | T1: Verify REGISTRATION period | T2: Fail period check | T3: Check time conflict | T4: Conflict detected | T5: Check capacity | T6: Capacity available | T7: Class full → waitlist | T8: Instructor promotes from waitlist | T9: Max-courses guard fires
INITIAL MARKING	Token in P0. Each registration attempt flows the token through T0→P1→T1. If period OK: T3 fires; if conflict free: T5 fires; capacity split: T6→P8 or T7→P9.

Element	Description
KEY LOGIC	T9 (max-courses guard) uses an inhibitor arc from a count place: if count=4 tokens in the enrolled-courses place, T0 is disabled. T8 (promote) consumes token from P9 and places it in P10 then P8, simultaneously decrementing the waitlist count place.



UC-05: Class Running Period Rules

Actors: Registrar, Instructor, Student

Preconditions: System has just transitioned to RUNNING. All students have finalised registrations.

Normal Scenario

- RunningPeriodJob executes automatically on transition to RUNNING.

- Step 1 — Under-enrolled student check: any student with fewer than 2 enrolled courses receives an automated warning.
- Step 2 — Under-enrolled course check: any course with fewer than 3 enrolled students is automatically cancelled (cancelled = true).
- Step 3 — Instructor notification: instructor of each cancelled course receives a formal warning.
- Step 4 — All-courses-cancelled check: if all of an instructor's courses are cancelled, instructor.suspendedNextSem = true.
- Step 5 — Special re-registration window: students from cancelled courses may register for any available open course.
- Step 6 — Window closes: no further enrolment changes are permitted until the next semester.

Exceptional Scenarios

- E1 — Student from cancelled course cannot find an alternative: they remain with their remaining courses (which may be below 2 — another warning is issued).
- E2 — All courses cancelled for an instructor: suspendedNextSem set; instructor cannot be assigned courses in the next SETUP period.
- E3 — Re-registration window conflict: standard conflict detection applies during the special window.

Postconditions

- Classes are running. All under-enrolment warnings issued. Cancelled courses removed from active schedule. Special re-registration complete.

Petri Net — UC-05: Running Period Job

> *This net models the automated RunningPeriodJob as a sequence of parallel checks. The join transition T_join requires all checks to complete before the special re-registration window opens.*

Element	Description
PLACES	P0: RUNNING triggered | P1: Student check pending | P2: Course check pending | P3: Students warned (< 2 courses) | P4: Courses cancelled (< 3 students) | P5: Instructor warned | P6: Instructor suspended next sem | P7: Re-reg window open | P8: Re-reg window closed | P9: Running period stable
TRANSITIONS	T0: Fork → parallel checks | T1: Warn under-enrolled students | T2: Cancel under-enrolled courses | T3: Warn instructors of cancelled courses | T4: Suspend instructor (all courses cancelled) | T_join: Synchronise all checks | T5: Open re-reg window | T6: Close re-reg window
INITIAL MARKING	Token in P0. T0 immediately forks one token into P1 and one into P2 (parallel firing). T1 and T2 fire independently. T3 and T4 are conditional on T2 results. T_join requires tokens in both post-T1 and post-T2 places before opening the re-reg window.
KEY LOGIC	T4 uses an inhibitor arc from a 'remaining-courses' place: if count > 0 for instructor, T4 is disabled. T4 only fires when ALL courses for that instructor have been cancelled. T_join is the

Element	Description
	synchronisation barrier ensuring both student and course checks complete before re-reg opens.

```
  P0(RUNNING) ──T0 (fork)──> P1(Student Check) ──T1──> P3(Students Warned)
                          └──> P2(Course Check)  ──T2──> P4(Courses Cancelled) ──T3──>
P5(Instructors Warned)

└─T4(inhibited if courses remain)──> P6(Suspend Next Sem)
  P3 + P5 ──T_join (sync barrier)──> P7(Re-Reg Window Open) ──T5──> P7 active ──T6──>
P8(Window Closed) ──> P9(Running Stable)
```

UC-06: Class Reviews and Star Ratings

Actors: Student (reviewer), Registrar (moderator and identity viewer)

Preconditions: System is in RUNNING period. Student is enrolled in the course.

Normal Scenario

- Student navigates to the course page and selects 'Write Review'.
- Student enters review text and selects a star rating (1–5).
- ReviewService.submit() is called.
- TabooFilter.check(text, tabooList) performs case-insensitive partial-match scan. Returns violation count and redacted text.
- Branch A — 0 violations: review published as-is; author identity visible only to Registrar.
- Branch B — 1 or 2 violations: review published with taboo words replaced by *; student receives 1 warning.
- Branch C — 3 or more violations: review not published; student receives 2 warnings.
- On publication: Course.avgRating is recalculated. If avgRating < 2.0, instructor receives 1 warning.
- If instructor accumulates 3 total warnings from any source: instructor is suspended from teaching.

Exceptional Scenarios

- E1 — Review submitted after instructor posts final grades: system blocks submission with an error.
- E2 — Student not enrolled in the course: system blocks with an enrolment-check error.
- E3 — Registrar updates taboo list: existing published reviews are not retroactively reprocessed.
- E4 — Instructor already at 2 warnings and avgRating drops below 2.0: the new warning triggers immediate suspension.

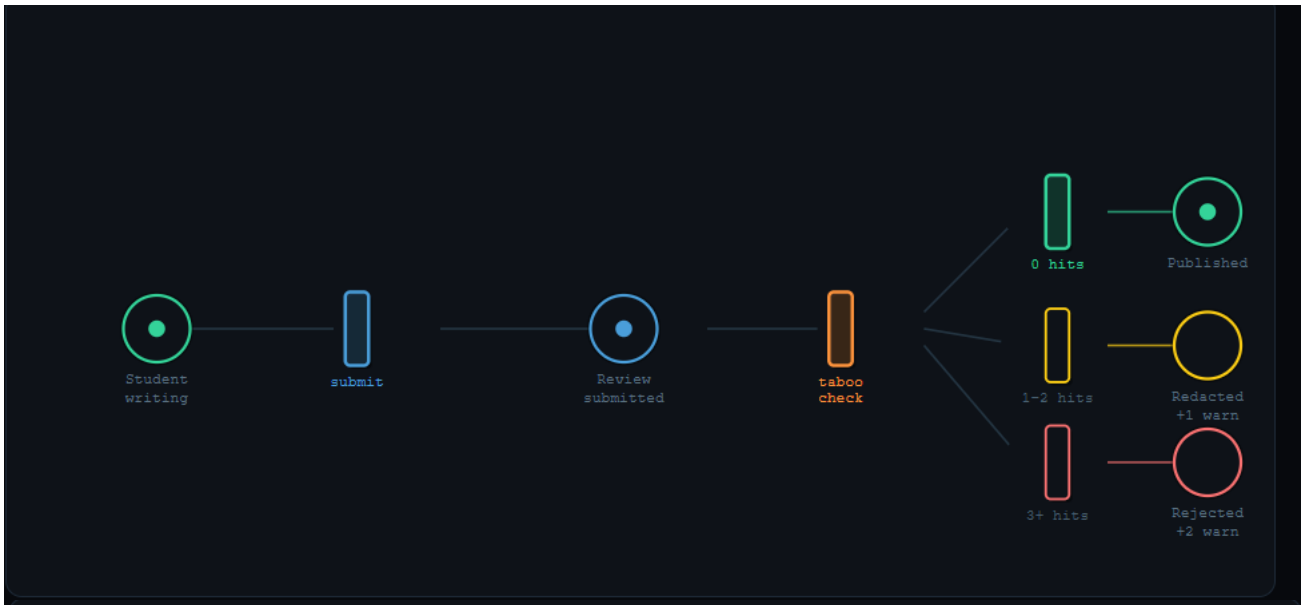
Postconditions

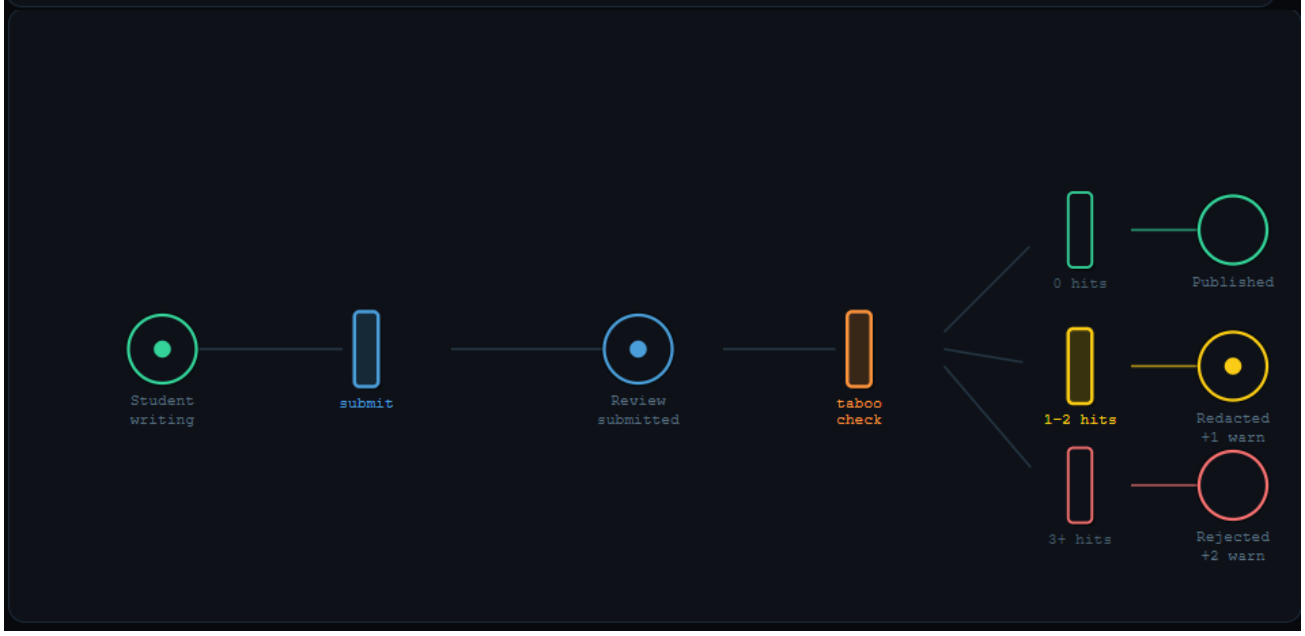
- Review is published (possibly redacted) or suppressed. Warnings issued as appropriate. Course avgRating updated.

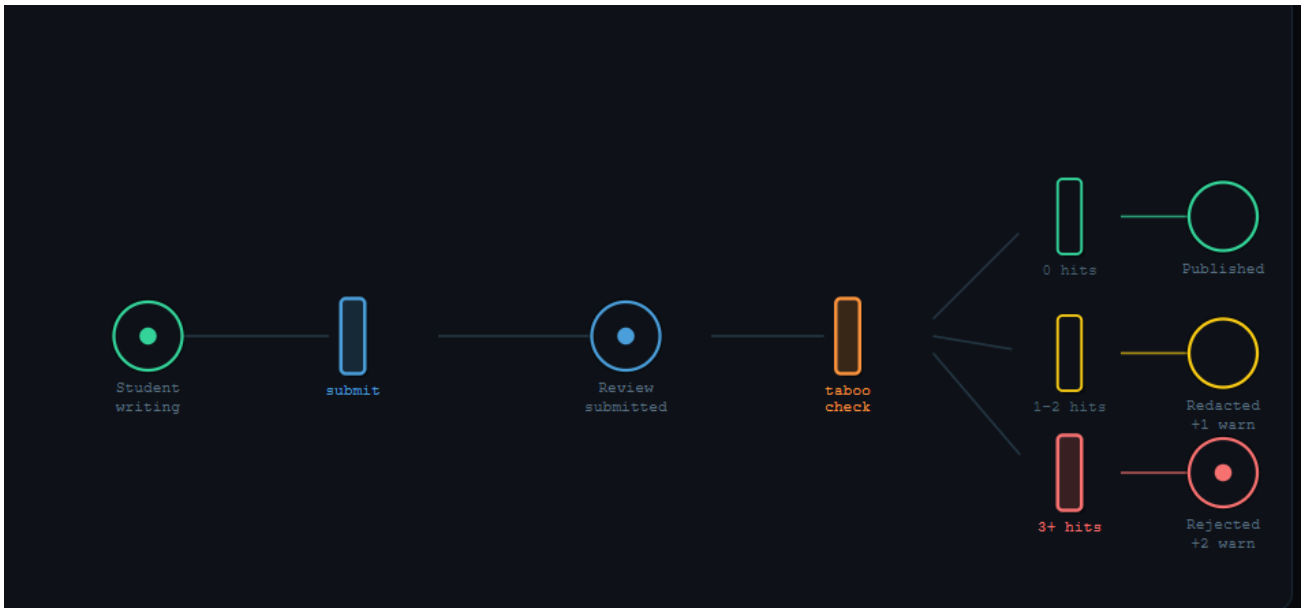
Petri Net — UC-06: Review Submission and Taboo Filter

> *This net captures the three-way branch based on taboo violation count, plus the downstream avgRating consequence.*

Element	Description
PLACES	P0: Student enrolled | P1: Review written | P2: Taboo scanned | P3: 0 violations | P4: 1-2 violations | P5: 3+ violations | P6: Published clean | P7: Published redacted | P8: Rejected | P9: 1 warning issued | P10: 2 warnings issued | P11: avgRating updated | P12: avgRating < 2.0 | P13: Instructor warned | P14: Instructor suspended
TRANSITIONS	T0: Submit review | T1: Run taboo filter | T2: 0-hit branch | T3: 1-2-hit branch (redact + warn) | T4: 3+-hit branch (reject + 2 warns) | T5: Publish clean | T6: Publish redacted | T7: Recalculate avgRating | T8: Rating OK (>= 2.0) | T9: Rating low (< 2.0) → warn instructor | T10: Instructor hits 3 warnings → suspend
INITIAL MARKING	Token in P0. T0 fires when student submits. T1 runs the filter. Based on violation count exactly one of T2/T3/T4 fires (conflict-free by guard conditions). T5 or T6 leads into P11; T7 then fires to update avgRating. T9 is enabled only when P12 holds a token.
KEY LOGIC	T3 is a multi-output transition: produces tokens in P7 (redacted review) AND P9 (1 warning). T4 produces tokens in P8 (rejected) AND P10 (2 warnings). T10 uses an inhibitor guard: only fires if instructor.warnings == 3, making it a threshold-triggered transition.


Student writing
submit
Review submitted
taboo check
0 hits
Published
1-2 hits
Redacted +1 warn
3+ hits
Rejected +2 warn


Student writing
submit
Review submitted
taboo check
0 hits
Published
1-2 hits
Redacted +1 warn
3+ hits
Rejected +2 warn



UC-07: Grading and Academic Standing

Actors: Instructor (grade submitter), Registrar (oversight), Student (recipient)

Preconditions: System is in GRADING period. Instructor is authenticated and assigned to the course.

Normal Scenario

- Instructor opens the grade submission interface and views all enrolled students per course.
- Instructor assigns a letter grade (A, B, C, D, or F) to each student.
- GradingService.submitGrades(courseId, grades[]) validates that every enrolled student has received a grade before accepting submission.
- At period close: ClosingSemesterJob fires.
- Step 1 — Ungraded check: any instructor who has not graded all students receives a warning.
- Step 2 — Class GPA audit: if class average > 3.5 or < 2.5, Registrar review window opens. No justification within window → instructor warned or dismissed.
- Step 3 — Student GPA recalculation: GpaEngine.recalcGPA(studentId) recomputes semGPA and cumulative GPA.
- Step 4 — Academic standing checks (in order): cumGPA < 2.0 → TERMINATED. Same course failed twice → TERMINATED. cumGPA 2.0–2.25 → warning + mandatory Registrar interview. semGPA > 3.75 or cumGPA > 3.5 (multi-semester) → Honor Roll.
- Step 5 — Honor Roll offset: each honor distinction can remove one active warning.
- Step 6 — Graduation check: student with 8 completed courses may apply; Registrar confirms.

Exceptional Scenarios

- E1 — Instructor dismissed: account deactivated immediately; grades already submitted are retained.

- E2 — Student terminated: account disabled; all records archived in read-only state.
- E3 — Honor Roll can offset only one warning per honor earned; multiple honors remove one-per-honor.
- E4 — Premature graduation application (fewer than 8 courses): student receives a warning.

Postconditions

- All grades recorded. Academic standing updated for all students and instructors. Terminations, suspensions, Honor Roll, and warnings applied. Semester status = CLOSED.

Petri Net — UC-07: Grading and Academic Standing

> *This is the most complex net in the system. ClosingSemesterJob fans out into multiple parallel standing checks, each of which may produce different outcome tokens.*

Element	Description
PLACES	P0: Grades submitted | P1: All students graded | P2: Ungraded students remain | P3: GPA recalculated | P4: cumGPA < 2.0 | P5: Double-fail detected | P6: cumGPA 2.0-2.25 | P7: GPA OK (> 2.25) | P8: Honor Roll eligible (semGPA > 3.75 or cumGPA > 3.5) | P9: TERMINATED | P10: Warning + interview | P11: Honor Roll awarded | P12: Warning removed (honor offset) | P13: 8 courses complete | P14: Graduated | P15: Class GPA outlier | P16: Instructor justification window open | P17: Instructor warned/dismissed
TRANSITIONS	T0: Submit grades | T1: Validate all graded | T2: Missing grades detected | T3: Recalculate GPA | T4: Terminate (cumGPA < 2.0) | T5: Terminate (double fail) | T6: Warn + schedule interview | T7: Standing OK | T8: Award Honor Roll | T9: Offset warning with honor | T10: Graduation confirmed | T11: Check class GPA outlier | T12: Open justification window | T13: No justification → warn/dismiss instructor
INITIAL MARKING	Token in P0 per course. T1 is enabled only when ALL enrollment records for the course have a non-null grade. T3 fans out: one token per student into GPA check places. T4, T5, T6, T7, T8 are mutually exclusive by guard (conflict-free by GPA range).
KEY LOGIC	T4 and T5 both lead to P9 (TERMINATED) — they are alternative paths to the same outcome. T8 (Honor Roll) and T9 (offset warning) are independent: T8 deposits token in P11, then T9 is enabled if P10 and P11 both hold tokens (honor + warning = cancel). T13 is a timed transition: fires after justification window expires without a token in the justification place.

```
  P0(Grades Submitted) ──T1(all graded?)──> P1(All Graded) ──T3(recalc GPA)──> P3(GPA
Ready)
                       └─T2(missing)    ──> Instructor Warning
  P3 ──T4(cumGPA < 2.0)   ──> P9(TERMINATED)
  P3 ──T5(double fail)    ──> P9(TERMINATED)
  P3 ──T6(GPA 2.0-2.25)  ──> P10(Warning + Interview)
  P3 ──T7(GPA > 2.25)    ──> P7(Standing OK)
  P3 ──T8(semGPA > 3.75) ──> P11(Honor Roll) ──T9(P10 + P11)──> P12(Warning Removed)
  P7 ──T_grad(8 courses) ──> P13 ──T10(Registrar confirms)──> P14(GRADUATED)
  Class GPA: T11(outlier?) ──> P15 ──T12──> P16(Window) ──T13(timeout)──> P17(Instructor
Warned/Dismissed)
```

UC-08: Complaints and Disciplinary Actions

Actors: Student (complainant or subject), Instructor (complainant or subject), Registrar (arbitrator)

Preconditions: Filer is authenticated. Subject is an active user.

Normal Scenario

- A student files a complaint against another student or an instructor by selecting the subject and entering a description.
- Alternatively, an instructor files against a student requesting a warning or de-registration.
- ComplaintService.file() persists the complaint with status = OPEN and adds it to the Registrar queue.
- Registrar investigates: reviews complaint, subject history, and any prior complaints.
- For student-initiated complaints: Registrar issues a warning to the appropriate party.
- For instructor-initiated complaints: Registrar must act — options are (a) issue warning to student, (b) de-register student from course, or (c) if complaint is without merit, warn the instructor instead.
- Cumulative warning tracking: any user reaching 3 total warnings is automatically suspended for one semester.
- Suspended students must pay a fine to the Registrar before reinstatement.
- All complaints, decisions, and warnings are written to the immutable audit log.

Exceptional Scenarios

- E1 — Multiple complaints against same subject: tracked cumulatively; warnings accumulate.
- E2 — Registrar attempts to close instructor-initiated complaint without action: system blocks closure, action is mandatory.
- E3 — Suspended student tries to log in: sees suspension notice and fine amount; all features locked.
- E4 — Complaint filed against the Registrar: system currently has no escalation path (out of scope for this version).

Postconditions

- Complaint resolved. Applicable warnings, suspensions, or de-registrations applied. Audit log updated.

Petri Net — UC-08: Complaint and Disciplinary Flow

> *This net models the dual-entry complaint path (student-filed vs instructor-filed) and the mandatory-action constraint on instructor complaints.*

Element	Description
PLACES	P0: Complaint filed (student) | P1: Complaint filed (instructor) | P2: In Registrar queue | P3: Under investigation | P4: Warning to student | P5: Warning to instructor | P6: Student

Element	Description
	de-registered | P7: Complaint dismissed (merit check fail → instructor warned) | P8: Subject warnings < 3 | P9: Subject warnings >= 3 | P10: Suspended | P11: Fine paid | P12: Reinstated | P13: Audit logged
TRANSITIONS	T0: Student submits complaint | T1: Instructor submits complaint | T2: Route to queue | T3: Registrar opens investigation | T4: Issue warning (student-initiated) | T5: Issue warning to student (instructor-initiated) | T6: De-register student (instructor-initiated) | T7: Dismiss complaint → warn instructor (without merit) | T8: Warning count < 3 | T9: Warning count reaches 3 → suspend | T10: Student pays fine | T11: Log to audit
INITIAL MARKING	Token placed in P0 (student complaint) or P1 (instructor complaint). Both merge into P2 via T2. T3 requires Registrar action token. For instructor-initiated path, T4 is disabled — only T5, T6, or T7 are enabled (mandatory-action constraint modelled via inhibitor on T_close).
KEY LOGIC	The mandatory-action constraint is modelled by an inhibitor arc from P2 to a T_close transition: T_close (close without action) is disabled while an instructor-complaint token is in P2. Only T5, T6, or T7 can remove the instructor-complaint token. T9 is a threshold transition guarded by warnings == 3.

```
  P0(Student Complaint) ──T0──\
                              ──T2──> P2(Registrar Queue) ──T3──> P3(Under
Investigation)
  P1(Instructor Complaint)──T1──/
  [Student-initiated path] P3 ──T4──> P4(Warning) ──T8(< 3)──> P8(OK)
                                                   └─T9(== 3)──> P9──>
P10(Suspended)──T10(fine)──> P11──> P12(Reinstated)
  [Instructor-initiated path] P3 ──T5──> P4(Warn Student)
                                 ──T6──> P6(De-register)
                                 ──T7──> P7(Dismiss + Warn Instructor)
  Inhibitor: P1 token ──|──> T_close  (cannot close instructor complaint without action)
  All outcomes ──T11──> P13(Audit Logged)
```

UC-09: User Personal Pages

Actors: Student, Instructor, Registrar

Preconditions: User is authenticated.

Normal Scenario

- On successful login, UserPageController.load(userId, role) routes to the role-appropriate view.
- New students (first login): TutorialService.run() executes a full interactive walkthrough before the personal page renders.
- Student view: academic record (all completed courses + grades), cumulative GPA, current registrations, active warnings, Honor Roll distinctions, program completion %.
- Instructor view: assigned courses, enrolled students per course with academic records, active warnings.

- Registrar view: full admin dashboard — all queues (application, complaint), semester control panel, system config (taboo words, quota, period management), full access to all records.
- All authenticated users can launch the AI assistant from their personal page.

Exceptional Scenarios

- E1 — Suspended student logs in: suspension notice and fine amount displayed; all features locked.
- E2 — Terminated student logs in: read-only archive view only.
- E3 — New student has not completed tutorial: system does not allow the personal page to load until the tutorial walkthrough is acknowledged.

Postconditions

- User is viewing their role-appropriate personal page with all current data displayed.

Petri Net — UC-09: Login and Personal Page Routing

> *This net models the login routing logic including the first-login tutorial gate and the suspended/terminated access blocks.*

Element	Description
PLACES	P0: Login attempted | P1: Credentials valid | P2: Credentials invalid | P3: User is SUSPENDED | P4: User is TERMINATED | P5: User is ACTIVE (first login) | P6: User is ACTIVE (returning) | P7: Tutorial running | P8: Tutorial complete | P9: Student page loaded | P10: Instructor page loaded | P11: Registrar dashboard loaded | P12: Suspension notice shown | P13: Archive view shown
TRANSITIONS	T0: Validate credentials | T1: Invalid → error | T2: Check user status | T3: Status = SUSPENDED | T4: Status = TERMINATED | T5: Status = ACTIVE, first login | T6: Status = ACTIVE, returning | T7: Run tutorial | T8: Tutorial acknowledged | T9: Route to Student | T10: Route to Instructor | T11: Route to Registrar
INITIAL MARKING	Token in P0. T0 checks credentials. On success, T2 reads status. Status is exclusive — exactly one of T3/T4/T5/T6 fires. For T5, token goes to P7 (tutorial) and cannot proceed to P9 until T8 fires.
KEY LOGIC	The first-login gate is modelled by an inhibitor arc: T9 (load student page) is disabled if firstLogin flag is set. Only after T8 (tutorial complete) clears the firstLogin place does T9 become enabled. T3 and T4 are terminal for the session — no forward path to any page exists from P3 or P4 except their respective notice views.

```
  P0(Login) ──T0(validate)──> P1(Valid) ──T2(check status)──> P3(SUSPENDED) ──T3──>
P12(Suspension Notice)
            └─T1(invalid) ──> error                        ──> P4(TERMINATED)──T4──>
P13(Archive View)
                                                            ──> P5(ACTIVE, first)──T5──>
P7(Tutorial)──T7──> P8(Done)──T8──> P9/P10/P11
                                                            ──> P6(ACTIVE, return)──T6──>
P9(Student) or P10(Instructor) or P11(Registrar)
  Inhibitor: firstLogin flag ──|──> T9 (student page)  (blocked until tutorial done)
```

UC-10: AI-Powered Q&A Assistant

Actors: All actors (Visitor, Student, Instructor, Registrar). Access level varies by role.

Preconditions: AI panel accessible. Local vector database is populated with College0-specific data.

Normal Scenario

- User opens AI assistant panel and enters a natural-language question.
- AIAssistant.query(text, userRole, userId) is called.
- RoleContextBuilder.build(userRole, userId) constructs a role-scoped prompt context:
  - Visitor: public course schedule and admission info only.
  - Student: own enrolled courses and personal academic record.
  - Instructor: own assigned courses and enrolled students.
  - Registrar: unrestricted access to all system data.
- VectorDB.semanticSearch(text) embeds the query and searches the local store.
- If confidence >= 0.75: return local answer with a green 'Verified' badge.
- If confidence < 0.75: forward to OllamaClient.chat(model='llama3', messages=[role-context, query]).
- Ollama response is returned with a visible yellow hallucination warning banner.
- Response is role-filtered: any data outside the user's scope is omitted from the answer.

Exceptional Scenarios

- E1 — Ollama API unavailable: error message shown in AI panel with a Retry button; all non-AI features continue operating normally.
- E2 — Query requests another student's private data (non-Registrar role): answer is anonymised or aggregated; private identity never exposed.
- E3 — VectorDB returns high-confidence result but it is outdated: system has no mechanism to detect staleness; warning in documentation recommends periodic VectorDB refresh.
- E4 — Ollama model not pulled (model not found): OllamaClient.chat() returns 404; system falls back to error state with model-not-found message.

Postconditions

- User receives a role-scoped answer. Source badge (local vs LLM) displayed. Hallucination warning shown when applicable.

Petri Net — UC-10: AI Query Pipeline

> *This net models the full two-stage retrieval pipeline with role-context injection, cache-hit/miss branching, Ollama call, and error handling.*

Element	Description
PLACES	P0: Query submitted | P1: Role context built | P2: VectorDB search running | P3: Cache hit (confidence >= 0.75) | P4: Cache miss (confidence < 0.75) | P5: Local answer ready | P6: Ollama request sent | P7: Ollama response received | P8: API unavailable | P9: Model not found | P10: Role filter applied | P11: Answer delivered (local) | P12: Answer delivered (LLM + hallucination warning) | P13: Error state (retry available)
TRANSITIONS	T0: Build role context | T1: Embed + semantic search | T2: Confidence >= 0.75 (cache hit) | T3: Confidence < 0.75 (cache miss) | T4: POST to Ollama /api/chat | T5: Ollama responds 200 OK | T6: Ollama unreachable (network error) | T7: Ollama 404 model not found | T8: Apply role filter to local answer | T9: Apply role filter + attach warning to LLM answer | T10: Deliver to user | T11: Show error + retry
INITIAL MARKING	Token in P0. T0 fires immediately (synchronous). T1 fires, placing token in P2. T2 and T3 are in conflict: exactly one fires based on the confidence guard. T4 fires from P4; T5/T6/T7 are in conflict on the Ollama response.
KEY LOGIC	T2 and T3 are a conflict pair: they share input place P2. Guard condition (confidence >= 0.75) makes them conflict-free. T5, T6, T7 share input P6 and are also conflict-free by HTTP status code guard. T8 and T9 both flow into P10 (role filter) ensuring filtering always happens regardless of source. T11 is a self-loop retry: places token back in P0.

```
  P0(Query) ──T0(role context)──> P1 ──T1(semantic search)──> P2(VectorDB searching)
  P2 ──T2(confidence >= 0.75)──> P3(Cache Hit) ──T8(role filter)──> P10 ──T10──>
P11(Answer, local badge)
  P2 ──T3(confidence < 0.75) ──> P4(Cache Miss) ──T4(POST Ollama)──> P6(Ollama Sent)
  P6 ──T5(200 OK)            ──> P7(Response) ──T9(role filter + warning)──> P10 ──T10──>
P12(Answer + ⚠️ hallucination warning)
  P6 ──T6(network error)     ──> P8(Unavailable) ──T11(retry)──> P0
  P6 ──T7(404 model missing)──> P9(Model Not Found) ──T11──> P13(Error shown)
```

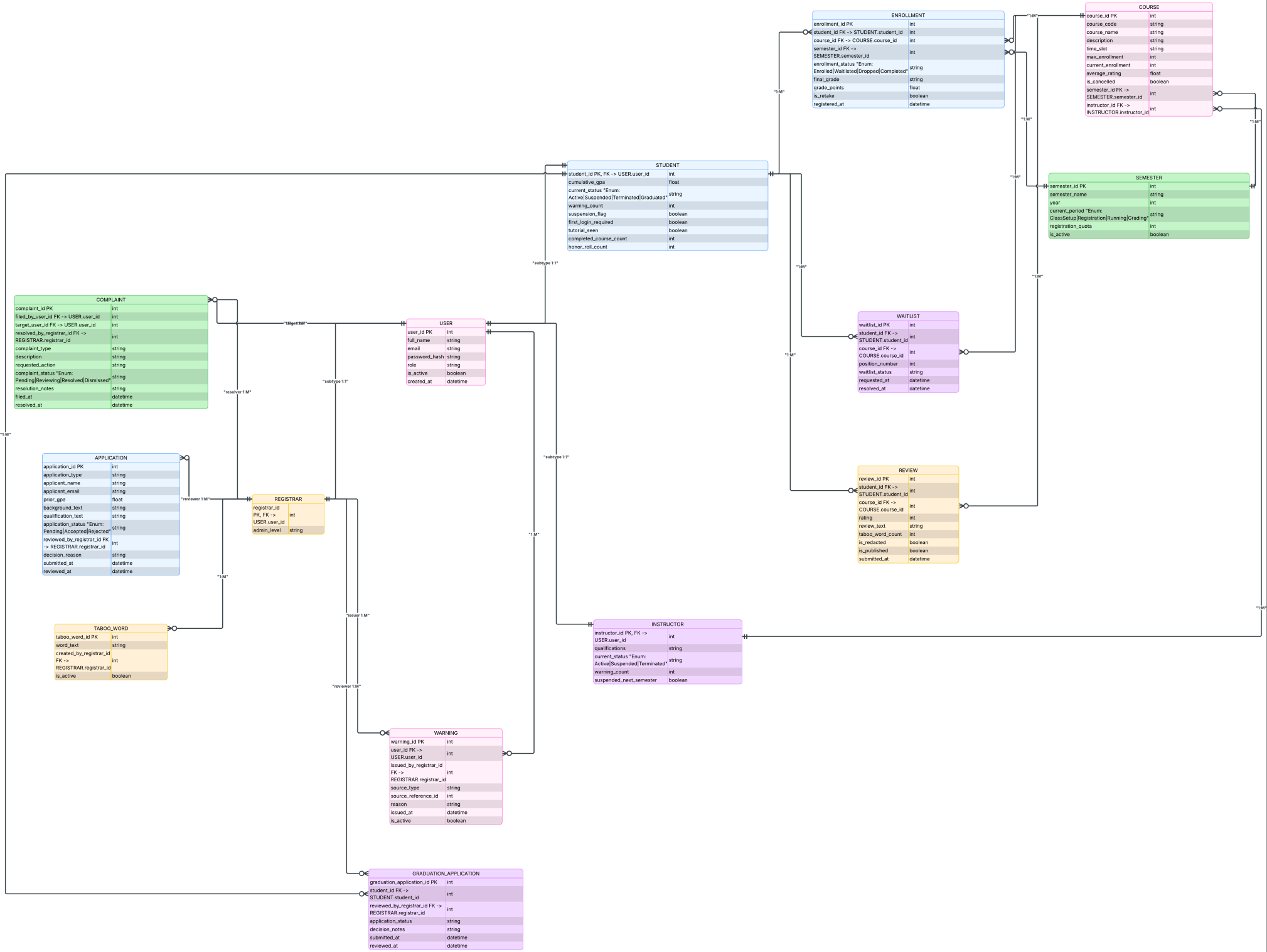
Summary: Use Case – Petri Net Cross-Reference

The table below maps each use case to the key Petri net elements and the most important behavioural properties modelled.

UC	Name	Key Places	Key Transitions	Notable Net Property
01	Public Dashboard	P3/P4/P5 (data fetched)	T4 join (sync barrier)	AND-join: all 3 fetches must complete
02	Visitor Application	P5 (eval), P11 (justification)	T5 override, T7 justify	Conditional mandatory field on override
03	Semester Periods	P0–P4 (period states)	T0–T3 advance; Terr	Sequential SM; inhibitor arcs prevent reversal
04	Course Registration	P4 (no conflict), P8/P9 (enrolled/wait)	T5/T7 capacity split	Inhibitor on max-courses count
05	Running Period Job	P3 (students warned), P4 (courses cancelled)	T_join sync barrier	Fork-join: parallel checks, single re-reg gate
06	Review & Taboo	P3/P4/P5 (violation count branches)	T2/T3/T4 conflict trio	Conflict-free branching on guard; threshold transition
07	Grading & Standing	P4/P5/P6/P8 (standing outcomes)	T4/T5 termination; T8/T9 honor	Most complex net; timed transition T13
08	Complaints	P2 queue; P10 suspension	T7 mandatory action (inhibited close)	Inhibitor enforces mandatory action rule
09	Personal Pages	P5 (first login), P7 (tutorial)	T5/T8 tutorial gate	Inhibitor gate: page blocked until tutorial done
10	AI Assistant	P3/P4 (hit/miss); P8/P9 (errors)	T2/T3 conflict pair; T11 retry self-loop	Conflict-free guard on confidence; self-loop retry

> *Total Petri net elements across all use cases: ~90 places, ~80 transitions, ~40 arc types including standard arcs, inhibitor arcs, and side-effect arcs.*

College0 ER Diagram



College0 – Phase II Design Report

Section 4: Detailed Design – Method Pseudo-code

Every public controller and service method is documented below with its inputs, outputs, preconditions, and step-by-step pseudo-code. Methods are grouped by class.

4.1 AuthController

Handles authentication, session management, and password operations.

authenticate(userId: String, plainPassword: String) → SessionToken | LoginError

```
INPUT:  userId       - the user's login ID
        plainPassword - plaintext password entered by user
OUTPUT: SessionToken on success, LoginError on failure
PRECONDITION: System is running; UserDAO is accessible.

1.  user ← UserDAO.findById(userId)
2.  IF user IS NULL:
        RETURN LoginError("INVALID_CREDENTIALS")
3.  IF NOT bcrypt.compare(plainPassword, user.passwordHash):
        RETURN LoginError("INVALID_CREDENTIALS")
4.  token ← generateSecureToken(userId, expiresIn = SESSION_TIMEOUT)
5.  SessionStore.save(token, userId, expiresAt = now() + SESSION_TIMEOUT)
6.  IF user.role == "STUDENT" AND user.firstLogin == true:
        RETURN { token, requirePasswordChange: true }
7.  RETURN { token, requirePasswordChange: false }
```

changePassword(token: SessionToken, newPassword: String) → void | ValidationError

```
INPUT:  token       - valid session token
        newPassword - new plaintext password chosen by user
OUTPUT: void (updates DB) or ValidationError
PRECONDITION: Token is valid and not expired.

1.  userId ← SessionStore.getUserId(token)
2.  IF isExpired(token):
        THROW ValidationError("SESSION_EXPIRED")
3.  IF NOT isStrongPassword(newPassword):
        THROW ValidationError("PASSWORD_TOO_WEAK")
        // Rules: min 8 chars, at least 1 digit, 1 uppercase
4.  hash ← bcrypt.hash(newPassword, SALT_ROUNDS = 12)
5.  UserDAO.update(userId, { passwordHash: hash, firstLogin: false })
6.  IF UserDAO.findById(userId).role == "STUDENT":
        TutorialController.launch(userId)
7.  RETURN
```

logout(token: SessionToken) → void

```
INPUT:  token - active session token
OUTPUT: void

1.  SessionStore.invalidate(token)
2.  RETURN
```

4.2 RegistrarController

Super-user operations: application processing, semester management, complaint resolution, taboo word management, and instructor oversight.

approveStudentApplication(appId: String) → Student | Error

```
INPUT:  appId - ID of the pending student application
OUTPUT: Student record or Error
PRECONDITION: Registrar is authenticated.

1.  app ← ApplicationDAO.findById(appId)
2.  IF app IS NULL OR app.status != "PENDING":
        THROW Error("APPLICATION_NOT_FOUND_OR_ALREADY_PROCESSED")
3.  quota ← StudentDAO.countActive()
4.  IF quota >= MAX_STUDENT_QUOTA:
        THROW Error("QUOTA_FULL")
5.  IF app.priorGPA <= 3.0:
        THROW Error("GPA_BELOW_THRESHOLD")
6.  studentId  ← generateUniqueId("S")
7.  tempPass   ← generateSecurePassword()
8.  student    ← StudentDAO.create({
            studentId,
            name:           app.name,
            email:          app.email,
            passwordHash:   bcrypt.hash(tempPass, 12),
            firstLogin:     true,
            cumulativeGPA:  0.0,
            semesterGPA:    0.0,
            warningCount:   0,
            enrollmentStatus: "ACTIVE"
        })
9.  ApplicationDAO.update(appId, { status: "ACCEPTED" })
10. NotificationService.sendCredentials(student, tempPass)
11. RETURN student
```

rejectStudentApplication(appId: String, justification: String) → void | Error

```
INPUT:  appId         - ID of the pending application
        justification - required if applicant GPA > 3.0 AND quota available
OUTPUT: void or Error
PRECONDITION: Registrar is authenticated.

1.  app ← ApplicationDAO.findById(appId)
2.  quota ← StudentDAO.countActive()
3.  IF app.priorGPA > 3.0 AND quota < MAX_STUDENT_QUOTA:
        IF justification IS NULL OR justification.trim() == "":
            THROW Error("JUSTIFICATION_REQUIRED")
4.  ApplicationDAO.update(appId, { status: "REJECTED", justification })
5.  NotificationService.notifyRejection(app.email)
6.  RETURN
```

approveInstructorApplication(appId: String) → Instructor

```
INPUT:  appId - ID of the pending instructor application
OUTPUT: Instructor record
PRECONDITION: Registrar is authenticated.

1.  app ← ApplicationDAO.findById(appId)
2.  instructorId ← generateUniqueId("I")
3.  tempPass     ← generateSecurePassword()
4.  instructor   ← InstructorDAO.create({
            instructorId,
            name:         app.name,
```

```
            email:        app.email,
            passwordHash: bcrypt.hash(tempPass, 12),
            suspended:    false,
            warningCount: 0,
            ratingWarnings: 0
        })
5.  ApplicationDAO.update(appId, { status: "ACCEPTED" })
6.  NotificationService.sendCredentials(instructor, tempPass)
7.  RETURN instructor
```

advanceSemesterPeriod() → void | BlockedError

```
INPUT:  (none - acts on the currently active Semester)
OUTPUT: void or BlockedError
PRECONDITION: Registrar is authenticated; active semester exists.

1.  semester ← SemesterDAO.getActive()
2.  period   ← semester.currentPeriod
    // PERIOD_SEQUENCE = [SETUP, REGISTRATION, RUNNING, GRADING]

3.  IF period == GRADING:
        incomplete ← GradeDAO.getInstructorsWithMissingGrades(semester.id)
        IF incomplete.length > 0:
            THROW BlockedError("MISSING_GRADES", incomplete)
        // Trigger post-grading jobs
        AcademicActionsJob.run()

4.  IF period == SETUP:
        IF CourseDAO.countForSemester(semester.id) == 0:
            THROW BlockedError("NO_COURSES_CONFIGURED")

5.  nextPeriod ← PERIOD_SEQUENCE[ indexOf(period) + 1 ]
6.  SemesterDAO.update(semester.id, { currentPeriod: nextPeriod })

7.  IF nextPeriod == RUNNING:
        CourseCancellationJob.run()

8.  NotificationService.broadcastPeriodChange(nextPeriod)
9.  RETURN
```

resolveComplaint(complaintId: String, action: String, targetId: String) → void

```
INPUT:  complaintId - ID of the complaint to resolve
        action      - one of: WARN_TARGET | WARN_FILER | DEREGISTER | DISMISS
        targetId    - user ID to act upon (where applicable)
OUTPUT: void
PRECONDITION: Registrar authenticated; complaint status is PENDING.

1.  complaint ← ComplaintDAO.findById(complaintId)
2.  SWITCH action:
        CASE "WARN_TARGET":
            WarningService.issueWarning(complaint.targetId, "COMPLAINT_UPHELD")
        CASE "WARN_FILER":
            WarningService.issueWarning(complaint.filerId, "UNJUSTIFIED_COMPLAINT")
        CASE "DEREGISTER":
            courseId ← complaint.referencedCourseId
            RegistrationDAO.drop(complaint.targetId, courseId)
            NotificationService.notify(complaint.targetId, "DEREGISTERED", courseId)
        CASE "DISMISS":
            // no action on users
3.  ComplaintDAO.update(complaintId, {
        status:     "RESOLVED",
        action:     action,
        resolvedBy: currentRegistrarId,
        resolvedAt: now()
```

```
    })
4.  RETURN
```

addTabooWord(word: String) → void

```
INPUT:  word - the word to prohibit in reviews
OUTPUT: void
PRECONDITION: Registrar authenticated.

1.  word ← word.trim().toLowerCase()
2.  IF TabooWordDAO.exists(word):
        RETURN  // already present; idempotent
3.  TabooWordDAO.create({ word, addedBy: currentRegistrarId, addedAt: now() })
4.  RETURN
```

removeTabooWord(wordId: String) → void

```
INPUT:  wordId - ID of the taboo word entry to remove
OUTPUT: void

1.  TabooWordDAO.delete(wordId)
2.  RETURN
```

fireInstructor(instructorId: String, reason: String) → void

```
INPUT:  instructorId - ID of the instructor to terminate
        reason       - written justification
OUTPUT: void
PRECONDITION: Registrar authenticated; instructor exists and is active.

1.  InstructorDAO.update(instructorId, {
        status: "TERMINATED",
        terminationReason: reason,
        terminatedAt: now()
    })
2.  courses ← CourseDAO.getTaughtBy(instructorId, activeSemester.id)
3.  FOR EACH course IN courses:
        // Treat as cancellation; students need replacement courses
        CourseCancellationJob.cancelSingle(course.id)
4.  NotificationService.notify(instructorId, "ACCOUNT_TERMINATED")
5.  RETURN
```

4.3 RegistrationController

Manages student course registration, waitlisting, and the special re-registration period.

register(studentId: String, courseIds: String[]) → Registration[] | ValidationError

```
INPUT:  studentId - enrolling student's ID
        courseIds - array of 2-4 course IDs chosen by the student
OUTPUT: array of Registration records (status ENROLLED or WAITLISTED)
PRECONDITION: Current period is REGISTRATION (or SPECIAL_REGISTRATION for affected
students).

1.  student ← StudentDAO.findById(studentId)
2.  IF student.enrollmentStatus == "SUSPENDED":
        THROW ValidationError("STUDENT_SUSPENDED")
3.  IF courseIds.length < 2 OR courseIds.length > 4:
        THROW ValidationError("INVALID_COURSE_COUNT",
              "Must select between 2 and 4 courses.")
4.  courses ← courseIds.map(id → CourseDAO.findById(id))
5.  // Time conflict detection
    allSlots ← courses.flatMap(c → c.timeSlots)
```

```
    IF hasDuplicates(allSlots):
        conflicting ← findConflictingPair(courses)
        THROW ValidationError("TIME_CONFLICT", conflicting)
6.  result ← []
7.  FOR EACH course IN courses:
        IF course.currentEnrollment < course.maxEnrollment:
            reg ← RegistrationDAO.create({
                studentId,
                courseId:   course.id,
                semesterId: activeSemester.id,
                status:     "ENROLLED",
                timestamp:  now()
            })
            CourseDAO.incrementEnrollment(course.id)
        ELSE:
            reg ← RegistrationDAO.create({
                studentId,
                courseId:   course.id,
                semesterId: activeSemester.id,
                status:     "WAITLISTED",
                timestamp:  now()
            })
        result.push(reg)
8.  RETURN result
```

dropCourse(studentId: String, courseId: String) → void

```
INPUT:  studentId, courseId
OUTPUT: void
PRECONDITION: Period is REGISTRATION; student is enrolled or waitlisted.

1.  reg ← RegistrationDAO.find(studentId, courseId, activeSemester.id)
2.  IF reg IS NULL: THROW Error("REGISTRATION_NOT_FOUND")
3.  IF reg.status == "ENROLLED":
        CourseDAO.decrementEnrollment(courseId)
        // Promote first waitlisted student if any
        next ← WaitlistDAO.getNext(courseId)
        IF next IS NOT NULL:
            RegistrationDAO.update(next.id, { status: "ENROLLED" })
            CourseDAO.incrementEnrollment(courseId)
            NotificationService.notify(next.studentId, "PROMOTED_FROM_WAITLIST",
courseId)
4.  RegistrationDAO.update(reg.id, { status: "DROPPED" })
5.  RETURN
```

4.4 ReviewController

Handles course review submission, taboo word filtering, and rating aggregation.

submitReview(studentId, courseId, text: String, rating: Int) → ReviewResult | Error

```
INPUT:  studentId - student submitting the review
        courseId  - course being reviewed
        text      - review body (free text)
        rating    - integer 1-5
OUTPUT: ReviewResult { status, warningsIssued } or Error
PRECONDITION: Period is RUNNING; student is enrolled; grades not yet posted.

1.  IF rating < 1 OR rating > 5:
        THROW Error("INVALID_RATING")
2.  IF GradeDAO.hasGradePosted(courseId, studentId):
        THROW Error("REVIEW_WINDOW_CLOSED")
```

```
3.  IF ReviewDAO.exists(studentId, courseId):
        THROW Error("REVIEW_ALREADY_SUBMITTED")

4.  taboo    ← TabooWordDAO.getAll().map(w → w.word)
    hits     ← scanForTaboo(text, taboo)   // returns list of matched words
    hitCount ← hits.length

5.  displayText ← text
    status     ← ""
    warnings   ← 0

6.  IF hitCount == 0:
        status ← "PUBLISHED"
    ELSE IF hitCount <= 2:
        displayText ← censorWords(text, hits)   // replace each hit with '*'
        status      ← "CENSORED"
        warnings    ← 1
    ELSE:
        status   ← "REJECTED"
        warnings ← 2

7.  IF status != "REJECTED":
        ReviewDAO.save({
            studentId, courseId, rating,
            text:       displayText,
            rawText:    text,         // visible only to Registrar
            status,
            timestamp:  now()
        })
        newAvg ← CourseDAO.recalcAvgRating(courseId)
        IF newAvg < 2.0:
            WarningService.issueWarning(
                CourseDAO.getInstructorId(courseId),
                "LOW_COURSE_RATING"
            )

8.  IF warnings > 0:
        WarningService.issueWarnings(studentId, warnings, "TABOO_WORDS")

9.  RETURN { status, warningsIssued: warnings }
```

4.5 GradeController

Manages grade submission during the Grading Period and triggers downstream GPA calculations.

```
submitGrade(instructorId, courseId, studentId, grade: String) → void | Error
INPUT:  instructorId - submitting instructor (must own the course)
        courseId     - course being graded
        studentId    - student receiving the grade
        grade        - one of: A | B | C | D | F
OUTPUT: void or Error
PRECONDITION: Current period is GRADING.

1.  course ← CourseDAO.findById(courseId)
2.  IF course.instructorId != instructorId:
        THROW Error("UNAUTHORIZED")
3.  IF grade NOT IN ["A","B","C","D","F"]:
        THROW Error("INVALID_GRADE")
4.  GradeDAO.upsert({ courseId, studentId, grade,
                      semesterId: activeSemester.id,
                      submittedAt: now() })
```

```
5.  AcademicRecordService.recalculate(studentId)

6.  // Check class average after every submission
    avg ← GradeDAO.classAverage(courseId)
    IF avg > 3.5 OR avg < 2.5:
        RegistrarNotificationDAO.save({
            type:      "CLASS_AVG_OUTLIER",
            courseId,
            avg,
            createdAt: now()
        })
7.  RETURN
```

checkGradingCompletion(semesterId: String) → InstructorId[]

```
INPUT:  semesterId
OUTPUT: list of instructorIds with incomplete grading

1.  instructors ← InstructorDAO.getActiveBySemester(semesterId)
2.  incomplete  ← []
3.  FOR EACH instructor IN instructors:
        courses ← CourseDAO.getTaughtBy(instructor.id, semesterId)
        FOR EACH course IN courses:
            ungraded ← RegistrationDAO.countUngraded(course.id)
            IF ungraded > 0:
                incomplete.push(instructor.id)
                BREAK
4.  RETURN incomplete
```

4.6 AcademicRecordService

Maintains each student's GPA and drives all post-grading academic status decisions.

recalculate(studentId: String) → void

```
INPUT:  studentId
OUTPUT: void (updates AcademicRecord in DB)

1.  allGrades  ← RegistrationDAO.getCompletedGrades(studentId)
    // 'completed' = status ENROLLED with a grade that is not NULL

2.  semGrades  ← allGrades.filter(g → g.semesterId == activeSemester.id)
3.  semGPA     ← semGrades.length > 0
                     ? average(semGrades.map(g → gradeToGPA(g.grade)))
                     : 0.0
    // gradeToGPA: A=4.0, B=3.0, C=2.0, D=1.0, F=0.0

4.  cumGPA     ← allGrades.length > 0
                     ? average(allGrades.map(g → gradeToGPA(g.grade)))
                     : 0.0

5.  completed  ← allGrades.filter(g → g.grade != "F").map(g → g.courseId)

6.  failedTwice ← allGrades
                     .groupBy(g → g.courseId)
                     .filter(group → group.filter(g → g.grade == "F").length >= 2)
                     .keys()

7.  AcademicRecordDAO.update(studentId, {
        semesterGPA:       semGPA,
        cumulativeGPA:     cumGPA,
        completedCourses:  completed,
        coursesFailedTwice: failedTwice,
```

```
        semestersCompleted: AcademicRecordDAO.get(studentId).semestersCompleted + 1
    })
8.  RETURN
```

runAcademicActions() → void [Post-Grading automated job]

```
INPUT:  (none - runs over all active students after Grading Period ends)
OUTPUT: void

1.  students ← StudentDAO.getAllActive()
2.  FOR EACH student IN students:
        record ← AcademicRecordDAO.get(student.id)

        // ── Termination checks (irreversible) ──────────────────────
        IF record.cumulativeGPA < 2.0:
            StudentDAO.terminate(student.id, "GPA_TOO_LOW")
            NotificationService.notify(student.id, "ACCOUNT_TERMINATED")
            CONTINUE

        IF record.coursesFailedTwice.length > 0:
            StudentDAO.terminate(student.id, "FAILED_SAME_COURSE_TWICE")
            NotificationService.notify(student.id, "ACCOUNT_TERMINATED")
            CONTINUE

        // ── Academic probation ─────────────────────────────────────
        IF record.cumulativeGPA >= 2.0 AND record.cumulativeGPA <= 2.25:
            WarningService.issueWarning(student.id, "ACADEMIC_PROBATION")
            RegistrarNotificationDAO.scheduleInterview(student.id)

        // ── Honor Roll ─────────────────────────────────────────────
        semHonor ← record.semesterGPA > 3.75
        cumHonor ← record.semestersCompleted >= 1 AND record.cumulativeGPA > 3.5
        IF semHonor OR cumHonor:
            HonorRollDAO.add(student.id, activeSemester.id)
            // Student may spend this credit to remove one warning
            // (student-triggered action via dashboard, not auto-applied)
            NotificationService.notify(student.id, "HONOR_ROLL_EARNED")
3.  RETURN
```

spendHonorRollCredit(studentId: String, warningId: String) → void | Error

```
INPUT:  studentId - student redeeming their credit
        warningId - ID of the warning the student wants removed
OUTPUT: void or Error
PRECONDITION: Student has at least one unspent Honor Roll credit.

1.  credits ← HonorRollDAO.getUnspent(studentId)
2.  IF credits.length == 0:
        THROW Error("NO_HONOR_ROLL_CREDIT")
3.  warning ← WarningDAO.findById(warningId)
4.  IF warning.userId != studentId:
        THROW Error("UNAUTHORIZED")
5.  WarningDAO.update(warningId, { active: false, removedViaHonorRoll: true })
6.  HonorRollDAO.markSpent(credits[0].id)
7.  StudentDAO.decrementWarningCount(studentId)
8.  RETURN
```

4.7 WaitlistController

Allows instructors to manually approve or decline waitlisted students for their courses.

approveStudent(instructorId, courseId, studentId) → void | Error

```
INPUT:  instructorId - must own the course
        courseId, studentId
OUTPUT: void or Error

1.  course ← CourseDAO.findById(courseId)
2.  IF course.instructorId != instructorId:
        THROW Error("UNAUTHORIZED")
3.  reg ← RegistrationDAO.findWaitlisted(courseId, studentId)
4.  IF reg IS NULL:
        THROW Error("STUDENT_NOT_ON_WAITLIST")
5.  RegistrationDAO.update(reg.id, { status: "ENROLLED" })
6.  CourseDAO.incrementEnrollment(courseId)
7.  NotificationService.notify(studentId, "WAITLIST_APPROVED", courseId)
8.  RETURN
```

declineStudent(instructorId, courseId, studentId) → void | Error

```
INPUT:  instructorId, courseId, studentId
OUTPUT: void or Error

1.  course ← CourseDAO.findById(courseId)
2.  IF course.instructorId != instructorId: THROW Error("UNAUTHORIZED")
3.  reg ← RegistrationDAO.findWaitlisted(courseId, studentId)
4.  IF reg IS NULL: THROW Error("STUDENT_NOT_ON_WAITLIST")
5.  RegistrationDAO.update(reg.id, { status: "WAITLIST_DECLINED" })
6.  NotificationService.notify(studentId, "WAITLIST_DECLINED", courseId)
7.  RETURN
```

4.8 GraduationController

Processes student graduation applications and archives graduating students.

submitApplication(studentId: String) → GraduationApplication | Error

```
INPUT:  studentId
OUTPUT: GraduationApplication record or Error
PRECONDITION: Student is active; has completed at least 8 courses.

1.  record ← AcademicRecordDAO.get(studentId)
2.  IF record.completedCourses.length < 8:
        THROW Error("INSUFFICIENT_COMPLETED_COURSES")
3.  IF GraduationApplicationDAO.hasPending(studentId):
        THROW Error("APPLICATION_ALREADY_PENDING")
4.  app ← GraduationApplicationDAO.create({
        studentId,
        submittedAt: now(),
        status:      "PENDING"
    })
5.  RETURN app
```

approve(registrarId, appId: String) → void | Error

```
INPUT:  registrarId, appId
OUTPUT: void or Error

1.  app    ← GraduationApplicationDAO.findById(appId)
2.  record ← AcademicRecordDAO.get(app.studentId)
3.  missing ← REQUIRED_COURSES.filter(r → r NOT IN record.completedCourses)
4.  IF missing.length > 0:
        // Registrar rejects; student gets a warning for reckless application
        GraduationApplicationDAO.update(appId, { status: "REJECTED" })
        WarningService.issueWarning(app.studentId, "RECKLESS_GRADUATION_ATTEMPT")
        RETURN
5.  GraduationApplicationDAO.update(appId, { status: "APPROVED" })
```

```
6.  DegreeDAO.create({ studentId: app.studentId, awardedAt: now(), type: "Bachelors"
})
7.  StudentDAO.archive(app.studentId)
8.  NotificationService.notify(app.studentId, "GRADUATED")
9.  RETURN
```

4.9 CourseCancellationJob

Automated job triggered at the start of the Class Running Period.

run() → void

```
INPUT:  (none)
OUTPUT: void
PRECONDITION: Semester just transitioned to RUNNING period.

1.  semester  ← SemesterDAO.getActive()
2.  courses   ← CourseDAO.getForSemester(semester.id)
    affectedStudents ← []

3.  FOR EACH course IN courses:
        IF course.currentEnrollment < MIN_ENROLLMENT (= 3):
            cancelSingle(course.id)
            enrolled ← RegistrationDAO.getEnrolledStudents(course.id)
            affectedStudents.addAll(enrolled.map(e → e.studentId))
            WarningService.issueWarning(course.instructorId, "COURSE_CANCELLED")

4.  // Check instructors whose ALL courses were cancelled
    instructors ← InstructorDAO.getActiveBySemester(semester.id)
    FOR EACH instructor IN instructors:
        allCancelled ← CourseDAO.allCancelledForInstructor(instructor.id,
semester.id)
        IF allCancelled:
            InstructorDAO.update(instructor.id, { suspendedNextSemester: true })
            NotificationService.notify(instructor.id, "SUSPENDED_NEXT_SEMESTER")

5.  // Open Special Registration for displaced students
    IF affectedStudents.length > 0:
        SemesterDAO.update(semester.id, {
            specialRegistrationOpen:   true,
            specialRegistrationStudents: affectedStudents
        })
        FOR EACH studentId IN affectedStudents:
            NotificationService.notify(studentId, "SPECIAL_REGISTRATION_OPEN")
6.  RETURN
```

cancelSingle(courseId: String) → void

```
INPUT:  courseId
OUTPUT: void

1.  CourseDAO.update(courseId, { status: "CANCELLED" })
2.  enrolled ← RegistrationDAO.getEnrolledStudents(courseId)
3.  FOR EACH enrollment IN enrolled:
        RegistrationDAO.update(enrollment.id, { status: "CANCELLED_BY_SYSTEM" })
4.  RETURN
```

4.10 WarningService

Central service for issuing warnings to students and instructors, and enforcing threshold-triggered consequences.

```
issueWarning(userId: String, reason: String) → Warning
INPUT:  userId - recipient of the warning
        reason - categorical reason code (e.g. TABOO_WORDS, LOW_COURSE_RATING)
OUTPUT: Warning record

1.  warning ← WarningDAO.create({
        userId,
        reason,
        sourceUCId:  currentUseCase,
        timestamp:   now(),
        active:      true
    })
2.  user  ← UserDAO.findById(userId)
3.  total ← WarningDAO.countActive(userId)

4.  IF user.role == "STUDENT":
        IF total >= 3:
            StudentDAO.suspend(userId, nextSemester.id)
            FineDAO.create({ studentId: userId, amount: FINE_AMOUNT, issuedAt: now()
})
            NotificationService.notify(userId, "SUSPENDED_WITH_FINE")

5.  IF user.role == "INSTRUCTOR":
        ratingWarnings ← WarningDAO.countByReason(userId, "LOW_COURSE_RATING")
        IF ratingWarnings >= 3:
            InstructorDAO.update(userId, { suspended: true })
            NotificationService.notify(userId, "SUSPENDED_RATING_WARNINGS")
        totalWarnings ← WarningDAO.countActive(userId)
        IF totalWarnings >= INSTRUCTOR_FIRE_THRESHOLD:
            InstructorDAO.terminate(userId)
            NotificationService.notify(userId, "ACCOUNT_TERMINATED")

6.  RETURN warning
```

```
issueWarnings(userId: String, count: Int, reason: String) → Warning[]
INPUT:  userId, count (number of warnings to issue), reason
OUTPUT: array of Warning records

1.  results ← []
2.  REPEAT count TIMES:
        w ← issueWarning(userId, reason)
        results.push(w)
3.  RETURN results
```

4.11 AIController

Manages the two-tier AI Q&A pipeline (local RAG → LLM fallback) and the Smart CRE explanation generator.

```
query(userId: String, role: String, question: String) → AIResponse |
ServiceError
INPUT:  userId   - querying user's ID (null for Visitor)
        role     - VISITOR | STUDENT | INSTRUCTOR | REGISTRAR
        question - natural language question
OUTPUT: AIResponse { text, source: "LOCAL"|"LLM", warning: Boolean }
```

```
1.  context ← buildRoleContext(userId, role)
    // VISITOR:    public course/admissions data only
    // STUDENT:    + enrolled courses, own academic record
    // INSTRUCTOR: + own courses, enrolled students' records
    // REGISTRAR:  all data

2.  embedding ← EmbeddingService.embed(question)
    // Uses local sentence-transformer model; no network required

3.  hits ← VectorDB.search(embedding, topK = 5, contextFilter = context)

4.  IF hits.maxScore >= SIMILARITY_THRESHOLD (= 0.75):
        answer ← composeFromChunks(hits.chunks)
        RETURN { text: answer, source: "LOCAL", warning: false }

5.  IF LLMGateway.isAvailable():
        raw ← LLMGateway.query(question, maxTokens = 500)
        RETURN {
            text:    raw,
            source:  "LLM",
            warning: true   // UI must display hallucination disclaimer
        }

6.  THROW ServiceError("AI_UNAVAILABLE")
```

`explainRecommendation(course: CourseDTO, record: AcademicRecord) → String`

```
INPUT:  course - course data (name, avgRating, failRate, requiredForGraduation)
        record - student's academic record (cumulativeGPA, completedCourses,
required)
OUTPUT: String (1-2 sentence natural-language explanation)

1.  prompt ← buildPrompt("""
        Student profile: GPA={record.cumulativeGPA},
        courses completed={record.completedCourses.length}/8,
        graduation requirements remaining={record.requiredRemaining}.
        Course: {course.name}, avg rating={course.avgRating},
        required for graduation={course.requiredForGraduation}.
        In 1-2 sentences, explain why this course is recommended.
        Be concise and student-friendly.
    """)
2.  IF NOT LLMGateway.isAvailable():
        RETURN buildFallbackExplanation(course, record)
        // e.g. "Highly rated course required for your degree."
3.  RETURN LLMGateway.query(prompt, maxTokens = 80)
```

4.12 SmartCREController

Generates personalised course recommendations for students during or before the Registration Period.

`getRecommendations(studentId: String) → RecommendationCard[]`

```
INPUT:  studentId
OUTPUT: ordered array of RecommendationCard (up to MAX_RECOMMENDATIONS = 6)
PRECONDITION: Student authenticated; course catalog is non-empty.

1.  record    ← AcademicRecordDAO.get(studentId)
2.  available ← CourseDAO.getAvailableNextSemester()
3.  IF available.length == 0:
        RETURN []  // UI shows "Recommendations available when registration opens"
```

```
4.  passed      ← record.completedCourses  // courseIds with grade != F
5.  failedOnce  ← record.allGrades
                      .filter(g → g.grade == "F")
                      .map(g → g.courseId)
    // Student may retake if failed only once; blocked if failed twice

6.  // Filter: remove courses already passed; keep retake-eligible failures
    eligible ← available.filter(c →
        c.id NOT IN passed
        AND (c.id NOT IN record.coursesFailedTwice)
    )

7.  required ← GRADUATION_REQUIREMENTS.filter(r → r NOT IN passed)

8.  // Priority 1: required graduation courses not yet passed
    priority1 ← eligible.filter(c → c.id IN required)
                        .sortBy(c → c.avgRating, DESC)

    // Priority 2: elective courses sorted by rating
    priority2 ← eligible.filter(c → c.id NOT IN required)
                        .sortBy(c → c.avgRating, DESC)

9.  IF record.cumulativeGPA < 2.5:
        priority2 ← priority2.filter(c → c.failRate < HIGH_FAIL_THRESHOLD)

10. ranked ← [...priority1, ...priority2]

11. // Remove courses that conflict with the top-already-selected batch
    //   (advisory conflict filtering — not hard enforcement)
    ranked ← advisoryFilterConflicts(ranked)

12. cards ← []
13. FOR EACH course IN ranked.slice(0, MAX_RECOMMENDATIONS):
        explanation ← AIController.explainRecommendation(course, record)
        cards.push(new RecommendationCard({
            course,
            explanation,
            isRequiredForGraduation: course.id IN required,
            isGPABooster: record.cumulativeGPA <= 2.25 AND course.avgGrade > 3.0
        }))

14. RETURN cards
```

4.13 TutorialController

Controls the new-student in-app tutorial lifecycle.

`launch(studentId: String, forced: Boolean = false) → void`

```
INPUT:  studentId - new student's ID
        forced    - true if manually re-launched from Help menu
OUTPUT: void (triggers UI event)

1.  state ← TutorialStateDAO.findByStudent(studentId)
2.  IF state IS NULL:
        state ← TutorialStateDAO.create({
            studentId,
            seen:      false,
            lastStep:  0
        })
3.  IF state.seen AND NOT forced: RETURN  // already completed; don't re-launch
```

```
4.  UI.showTutorialOverlay(TUTORIAL_STEPS)
    // Steps: Dashboard nav → Register for courses → Write a review →
    //        Use AI Q&A → View academic record
5.  RETURN
```

markComplete(studentId: String) → void

```
INPUT:  studentId
OUTPUT: void

1.  TutorialStateDAO.update(studentId, {
        seen:        true,
        completedAt: now(),
        lastStep:    TUTORIAL_STEPS.length
    })
2.  RETURN
```

— End of Section 4 —

PART 6: Memos of Group Meetings

Team Members

Jeremias
Giovanni
Diego
Munif
Stephan

Work Distribution Overview

The team divided responsibilities across all required sections of the design report, including system modeling, data design, interface design, and implementation planning. While certain members were assigned primary ownership of specific sections, collaboration was required throughout due to dependencies between components such as use cases, the ER diagram, and the system prototype.

Jeremias focused on the early conceptual phases of the project, including the system introduction and the development of use cases and scenarios. His work established the functional requirements and ensured alignment with the project specification.

Giovanni was primarily responsible for the design and implementation of the system screens and the working prototype. He also contributed to the ER diagram and managed the GitHub repository, including commit organization and version control.

Diego contributed to system logic and administrative functionality, including registrar-related behaviors and system constraints. He also played a role in validating the consistency between use cases, the ER diagram, and system rules.

Munif contributed to system flow and structural design, including navigation, interaction flow, and assisting with the ER diagram and relationships between entities.

Stephan contributed across all stages of development, including modeling, UI implementation, debugging, and integration. His role involved supporting multiple components to ensure overall system consistency.

Meeting Memo 1: April 15, 2026

Focus: Project scope, system overview, and planning

The team began by carefully reviewing the project specification in order to fully understand all requirements for the Phase II design report. This initial discussion focused on identifying each

required section, including the introduction, use cases, ER diagram, detailed design with pseudocode, system screens, and documentation components such as meeting memos and repository submission.

A significant portion of the meeting was dedicated to clarifying the scope of the system and ensuring that all members shared a consistent understanding of the expected functionality. The team discussed the concept of the College0 system as a simulated college management platform and identified the primary actors involved, including Registrar, Instructor, Student, and Visitor. For each role, the team began outlining expected responsibilities, permissions, and interactions within the system.

In addition, the team discussed how the different sections of the report are interconnected. For example, it was noted that use cases would directly influence both the ER diagram and the system screens, while the ER diagram would define the underlying data structure needed to support those interactions. This led to an emphasis on maintaining consistency across all components from the beginning of the project.

The team also discussed tools that would be used throughout development. Diagramming tools such as draw.io were considered for creating the ER diagram and other visual representations, while Figma was discussed as a potential tool for designing system screens and building a prototype. GitHub was selected as the platform for version control and collaboration, with an understanding that commits would reflect the progression of work over time.

Finally, the team began discussing how to divide responsibilities among members based on the different sections of the report. Although initial roles were outlined, it was acknowledged that collaboration would still be necessary due to dependencies between components.

Concerns:
Ensuring that all team members maintain a consistent and accurate understanding of the project requirements as development progresses
Potential overlap between different sections of the report, particularly between use cases, ER diagram design, and system screen implementation
Difficulty in coordinating early decisions across multiple components that depend on one another
Risk of misalignment if documentation and implementation are developed independently without regular communication

Meeting Memo 2: April 16, 2026

Focus: Use cases and system behavior

The team focused on developing a complete and accurate set of use cases for the College0 system. Each use case was reviewed in detail to ensure that it properly captured the intended

functionality of the system and aligned with the project specification. The team worked on defining both normal flows and exceptional scenarios for each use case, emphasizing clarity and completeness.

During this process, the team carefully analyzed how each actor interacts with the system. For example, the behaviors of Students, Instructors, Registrars, and Visitors were broken down into specific actions such as registration, course management, application submission, and system navigation. These interactions were then structured into step-by-step flows to ensure that all possible outcomes were considered.

Particular attention was given to identifying edge cases and constraints. This included handling invalid inputs, enforcing system rules such as course registration limits, managing permission restrictions between different user roles, and accounting for system-triggered events such as warnings, suspensions, or automatic actions. The team recognized that properly defining these scenarios was essential for ensuring that the system behaves predictably under all conditions.

The team also discussed how these use cases would later be translated into sequence diagrams or collaboration diagrams. It was emphasized that the use cases must be detailed enough to support these diagrams without requiring major revisions. As a result, the team made an effort to keep the structure of each use case consistent and logically organized.

In addition, the team ensured that use cases remained aligned with other parts of the project, particularly the ER diagram and system screens. It was noted that inconsistencies between these components could lead to confusion or errors later in development.

Concerns:
Ensuring that all use cases are complete and cover every required system interaction
Difficulty in clearly defining exceptional scenarios for more complex behaviors, especially those involving multiple conditions or system constraints
Maintaining consistency between use cases and other components such as the ER diagram and system screens
Avoiding overly simplistic descriptions that fail to capture important system logic

Meeting Memo 3: April 17, 2026

Focus: ER diagram planning and data modeling

The team focused on planning the structure of the ER diagram for the College0 system. This involved identifying all necessary entities required to accurately represent the system's data. Core entities such as User, Student, Instructor, Registrar, Course, Enrollment, Review, Complaint, and Application were discussed and justified based on the use cases developed in the previous meeting.

The team carefully analyzed how each entity relates to others within the system. Special attention was given to identifying and correctly modeling many-to-many relationships, particularly the relationship between students and courses. The team agreed to resolve these relationships using associative entities such as Enrollment, which would also store additional attributes like grades and enrollment status.

In addition, the team discussed how to handle role-based relationships within the system. Since multiple user roles share common attributes, the possibility of using a generalized User entity with specialized subtypes (Student, Instructor, Registrar) was considered to reduce redundancy and improve clarity in the data model.

The team also began defining attributes for each entity, including primary keys and relevant fields needed to support system functionality. For example, entities such as Course required attributes related to scheduling and capacity, while Review required attributes for ratings and text content. The importance of clearly defining primary keys and foreign key relationships was emphasized to ensure referential integrity within the system.

Tool selection was also discussed during this meeting. The team agreed to use a diagramming tool such as draw.io to construct the ER diagram, as it provides flexibility for organizing entities, relationships, and attributes in a clear and structured format. The team also discussed the importance of maintaining readability and organization within the diagram to ensure it is easy to understand.

Finally, the team ensured that the ER diagram design remained consistent with the use cases previously developed. It was emphasized that the data model must support all system behaviors, including registration, reviews, complaints, and administrative actions.

Concerns:
Ensuring that all entities include the correct attributes and properly defined primary keys
Maintaining consistency between the ER diagram and the use cases to avoid mismatches in system logic
Correctly modeling complex relationships, particularly many-to-many relationships and role-based structures
Avoiding redundancy in the data model while still capturing all necessary system information

Meeting Memo 4: April 18, 2026

Focus: ER diagram development and validation

During this meeting, the team focused on fully developing and refining the ER diagram based on the structure that was planned previously. The goal was to translate the conceptual model into a complete and accurate representation of the system's data. Each entity was revisited to ensure

that all required attributes were included and that primary keys were clearly defined and appropriate for identifying each record.

The team carefully reviewed relationships between entities, paying close attention to cardinality and participation constraints. For example, relationships such as student enrollment in courses were verified to ensure correct many-to-many modeling through associative entities. Foreign key connections were also examined to confirm that all relationships were properly implemented and that referential integrity would be maintained within the system.

In addition, the team evaluated whether the structure of the ER diagram accurately reflected the system behaviors described in the use cases. Each major use case, such as course registration, review submission, complaint handling, and application processing, was mapped back to the data model to ensure that all required interactions were supported by the diagram.

The team also discussed normalization and redundancy within the data model. Efforts were made to avoid unnecessary duplication of data while still maintaining clarity and completeness. Adjustments were made to certain entities and relationships to improve the overall structure and reduce potential inconsistencies.

Throughout the meeting, emphasis was placed on ensuring that the ER diagram was not only functionally correct but also clearly organized and easy to interpret. The layout of the diagram was adjusted to improve readability, grouping related entities together and minimizing overlapping relationships.

Concerns:
Avoiding missing relationships or incorrectly defined cardinality, which could lead to inconsistencies in the system
Ensuring that the ER diagram fully and accurately represents all system logic described in the use cases
Maintaining proper foreign key relationships to preserve data integrity
Balancing normalization with clarity to avoid making the diagram overly complex or difficult to understand

Meeting Memo 5: April 19, 2026

Focus: Detailed design and pseudocode

During this meeting, the team focused on the detailed design section of the project, specifically the development of pseudocode for key system functionalities. The objective was to translate the previously defined use cases and system behaviors into structured logical steps that clearly describe how the system operates internally.

The team began by identifying the most important system methods that required pseudocode representation. These included core functionalities such as user authentication, course registration, waitlist handling, review submission, and administrative actions. Each function was broken down into smaller steps to ensure clarity and logical flow.

For each method, the team discussed the necessary inputs, expected outputs, and internal processing steps. This included defining validation checks, conditional logic, and system constraints. For example, the course registration process required validation of course limits, time conflicts, and enrollment capacity. Similarly, review submission required handling of text input and filtering of restricted content.

The team emphasized the importance of writing pseudocode that is both clear and structured. A consistent format was discussed, including the use of indentation, control structures such as loops and conditionals, and clear naming of variables and actions. This was done to ensure that all pseudocode sections would be easy to read and understand.

Additionally, the team ensured that the pseudocode remained consistent with both the use cases and the ER diagram. Each method was reviewed to confirm that it accurately reflected the system's data structure and supported the defined interactions between entities.

Concerns:
Ensuring that pseudocode is detailed enough to clearly represent system behavior without becoming overly complex or difficult to follow
Maintaining consistency in formatting and structure across different pseudocode sections written by different team members
Avoiding logical gaps or missing steps that could lead to incomplete representation of system functionality
Ensuring alignment between pseudocode, use cases, and the ER diagram to maintain overall system consistency

Meeting Memo 6: April 20, 2026

Focus: System screens and prototype planning

During this meeting, the team transitioned from system modeling and design into the development of the user interface and working prototype. The primary objective was to determine how the system would be visually represented while still accurately reflecting the functionality defined in earlier stages, including use cases, the ER diagram, and pseudocode.

The team agreed to implement a web-based prototype to simulate the behavior of the College0 system. Although the actual system is described as a single-window desktop-style application, a web-based approach was chosen because it allows for easier demonstration of screens,

navigation, and interactions. The goal was not to build a fully functional system, but rather to create a realistic and structured representation of how the system would look and behave.

Tool selection was finalized during this meeting, with Figma being chosen as the primary tool for designing system screens and prototyping interactions. The team discussed how to organize the Figma workspace, including separating pages by system roles and grouping related screens together. Layout consistency was emphasized, with all screens following a unified structure consisting of a sidebar, header, and main content area.

The team began planning out the specific screens required for each role, including Student, Instructor, and Registrar dashboards. For the student role, particular attention was given to the course registration interface, as it represents one of the core functionalities of the system. The team ensured that screens would clearly reflect system constraints such as course limits, time conflicts, and waitlist behavior.

In addition, the team discussed how to visually represent system states and conditions, such as warnings, suspensions, and course availability. It was agreed that visual indicators such as badges, labels, and messages would be used to communicate these conditions clearly to the user.

Throughout the discussion, the team emphasized the importance of maintaining consistency between the prototype and previously defined system logic. The interface needed to accurately reflect the behavior described in use cases and pseudocode, rather than simply focusing on visual design.

Concerns:
Ensuring that the prototype accurately represents system rules and constraints defined in earlier stages
Balancing visual design with functional clarity to avoid overly complex or misleading interfaces
Maintaining consistency across screens designed by different team members
Ensuring that all required system features are clearly demonstrated within the prototype
Managing time effectively given the increased complexity of designing multiple role-based interfaces

Meeting Memo 7: April 21, 2026

Focus: Prototype development and refinement

During this meeting, the team continued the development of the system prototype and focused on improving the overall quality and consistency of the user interface. The primary objective was to refine previously designed screens and ensure that all components followed a unified layout and design standard.

The team reviewed each role-based interface, including Student, Instructor, and Registrar views, to verify that navigation, structure, and visual hierarchy were consistent across the system. Adjustments were made to spacing, alignment, and component sizing to improve readability and usability. In particular, attention was given to ensuring that text, buttons, and data displays were clearly visible and easy to interpret.

A significant portion of the meeting was dedicated to reviewing key system flows within the prototype. The course registration process was examined in detail to confirm that it accurately reflects system rules such as course limits, time conflicts, and waitlist behavior. The team ensured that users could clearly understand each step of the process through the interface.

In addition, minor bugs and inconsistencies were identified and corrected. These included issues with layout alignment, inconsistent component usage, and unclear labeling. The team worked collaboratively to resolve these issues and improve the overall presentation of the system.

The importance of maintaining consistency between the prototype and earlier design components, such as use cases and pseudocode, was emphasized. The team ensured that all screens continued to reflect the intended system behavior.

Concerns:
Limited time available to fully refine all system screens and address all minor inconsistencies
Ensuring consistent design and layout across interfaces developed by different team members
Difficulty in identifying all remaining issues within the prototype before final submission
Balancing the need for visual polish with the requirement to accurately represent system functionality

Meeting Memo 8: April 22, 2026

Focus: Integration and final preparation

During the final meeting, the team focused on integrating all components of the project and preparing the design report for submission. This included reviewing and finalizing all sections, such as use cases, the ER diagram, pseudocode, and system screens, to ensure that they were complete and properly aligned with one another.

The team verified that all required deliverables were included and met the assignment guidelines. Special attention was given to ensuring consistency across different sections of the report. For example, the team checked that system behavior described in use cases was accurately reflected in both the ER diagram and the prototype.

The GitHub repository was also reviewed and updated during this meeting. The team ensured that all files were properly organized and that commit history accurately reflected the progression

of work over time. Version control practices were discussed to confirm that the repository met submission requirements.

Final checks were performed to identify any missing components, formatting issues, or inconsistencies. The team worked together to resolve any remaining issues and ensure that the project was complete and ready for submission.

Concerns:
Limited time available for comprehensive testing and validation of all system features
Potential inconsistencies between different sections of the report due to parallel development
Risk of minor errors or omissions remaining undetected before submission
Challenges in fully verifying integration between independently developed components

Overall Team Concerns

The team experienced challenges in coordinating work across multiple sections of the project, particularly due to dependencies between use cases, the ER diagram, and system design. Ensuring consistency across all components required continuous communication.

Additionally, uneven workload distribution occurred due to the varying complexity of tasks. Some sections, such as the prototype and system screens, required significantly more effort than others.

Time constraints limited the ability to fully test and refine all aspects of the project. Despite these challenges, the team was able to complete all required components and produce a cohesive design report.

The address to the **GITHUB** repo:

https://github.com/itsjustgiio/college0-prototype